

RAPPORT FINAL

Analyse de conversations patient-thérapeute sur la santé mentale

Cahier des charges, Méthodologie,
Résultats et Analyses

Encadrante : Lynda Tamine-Lechani

Équipe projet :

*Ulysse Chasseigne, Idir Yahiaoui, Bantsoukissa Laure,
Cruz Fernandes Diogo, Emin Belkheir, Ezzo-Y-N Trésor PANA,
Logan Larroux, Yvan Sellier, Victor Blanquart-Blanchet, Lucas Da Silva*

1^{er} juin 2026



[Info5.BE-SHS]

Résumé

Ce rapport présente l'ensemble des travaux réalisés dans le cadre du projet d'analyse de conversations patient-thérapeute sur la santé mentale. Le projet s'articule autour de deux axes principaux : l'analyse exploratoire et la modélisation thématique des conversations (Étape 1), puis le développement d'un système d'analyse de sentiments et de question-réponse (Étape 2).

L'Étape 1 a permis d'explorer en profondeur le corpus de conversations à travers des analyses statistiques, lexicales et quatre méthodes de *topic modeling* (lexique manuel, TF-IDF + K-Means, Word Embeddings et LDA). Les résultats montrent que les thématiques principales tournent autour des relations interpersonnelles, de l'anxiété et de la dépression.

L'Étape 2 a vu le développement parallèle d'un classificateur de sentiments (méthodes supervisées et non supervisées, incluant des approches BERT avancées) et d'un système de question-réponse basé sur la similarité sémantique. Les modèles BERT avec fine-tuning atteignent une accuracy de 83,43%, surpassant significativement les méthodes classiques. Le système question-réponse, utilisant les représentations BERT, obtient un score de similarité de 0,702.

Mots-clefs

Analyse de sentiments - Topic Modeling - BERT - NLP - Santé mentale -
Système question-réponse - Word Embeddings - LDA - TF-IDF

Table des matières

Résumé	1
I Cahier des charges et organisation	7
1 Contexte et objectifs du projet	8
1.1 Présentation générale	8
1.2 Ressources utilisées	8
2 Organisation et gouvernance du projet	9
2.1 Organisation des groupes	9
2.2 Définition des rôles	9
2.2.1 Organisation étape 1 : Exploration du jeu de données	9
2.2.2 Organisation étape 2 : Développement du système d'analyse de sentiments et de question-réponse	10
II Étape 1 — Exploration du jeu de données	11
3 Méthodologie générale	12
4 Pré-traitement des données	13
4.1 Nettoyage — Structure générale	13
4.1.1 Tokenisation	13
4.1.2 Suppression de la ponctuation et des mots vides	14
4.1.3 Lemmatisation	14
4.2 Nettoyage — Approches comparées	15
4.2.1 Approche minimaliste (Emin)	15
4.2.2 Approche étendue (Diogo)	15
4.3 Résultats	16
4.3.1 Nombre de patients uniques	16
4.3.2 Nombre de thérapeutes uniques	16
4.3.3 Nombre de mots moyen par patient / thérapeute	17
5 Analyse lexicale et statistique	18
5.1 Mots les plus fréquents	18
5.1.1 Sans filtrage renforcé	18
5.1.2 Avec filtrage renforcé	18

5.1.3	Interprétation	18
5.2	Analyse lexicale — N-grams	19
5.2.1	Définition	19
5.2.2	Choix du n optimal	19
5.2.3	Résultats — Logan, Victor et Lucas	21
6	Topic Modeling	22
6.1	Préambule	22
6.2	Méthode 1 : Lexique personnalisé	23
6.2.1	Principe	23
6.2.2	Approche minimaliste (Trésor)	23
6.2.3	Approche étendue (Diogo)	23
6.2.4	Résultats	24
6.2.5	Évaluation critique	24
6.3	Méthode 2 : TF-IDF + K-Means	25
6.3.1	Principe de TF-IDF	25
6.3.2	Sélection du nombre de clusters (K)	26
6.3.3	Résultats selon K	27
6.3.4	Évaluation critique	29
6.4	Méthode 3 : Word Embeddings	30
6.4.1	Principe du Word Embedding	30
6.4.2	Skip-Gram vs CBOW	30
6.4.3	Word2Vec (Skip-gram) — Logan	31
6.4.4	Sentence Transformer (BERT / all-MiniLM-L6-v2) — Idir	32
6.4.5	spaCy (embeddings moyennés) — Emin	33
6.4.6	Bilan comparatif — Méthode 3	33
6.5	Méthode 4 : LDA (Latent Dirichlet Allocation)	35
6.5.1	Qu'est ce que le LDA ?	35
6.5.2	Résultats à 6 topics — Logan	36
6.5.3	Résultats à 6 topics — Diogo	36
6.5.4	Résultats à 5 topics — Lucas	37
6.5.5	Évaluation critique	37
6.6	Comparaison des méthodes	38
7	Bilan de l'étape 1	39
7.1	Problèmes techniques identifiés	39
7.2	Conclusion globale	40

III	Étape 2 — Analyse et Accès aux conversations	41
8	Vue d'ensemble	42
8.1	Étape 2 — Analyse et Accès aux conversations	42
8.1.1	Module 1 — Classification automatique des sentiments	42
8.1.2	Module 2 — Moteur de question-réponse orienté santé mentale . . .	43
9	Module 1 : Analyse des sentiments — Approches classiques	45
9.1	Contexte et organisation	45
9.1.1	Différence entre méthode supervisée et non-supervisée	45
9.1.2	Métriques d'évaluation utilisées	46
9.2	Résultats par membre	49
9.2.1	Méthode supervisée — Naive Bayes et Logistic Regression	49
9.2.2	Méthode non-supervisée — K-Means (Emin et Victor)	58
9.3	Comparaison globale : supervisé vs. non-supervisé	63
10	Module 1 EXTRA : Analyse des sentiments — Approches BERT	64
10.1	Qu'est-ce que BERT ?	64
10.1.1	Définition générale	64
10.1.2	Étapes d'apprentissage	65
10.1.3	Points forts de BERT	66
10.1.4	Limites de BERT	66
10.2	Trois variantes BERT implémentées	67
10.3	Initialisation commune aux trois modèles	68
10.3.1	Dépendances	68
10.3.2	Préparation des données	68
10.3.3	Découpage des données	69
10.3.4	Encodage des labels	69
10.4	Modèle A — BERT sans fine-tuning (extraction de features)	70
10.4.1	Principe et architecture	70
10.4.2	Tokenisation BERT	70
10.4.3	Extraction des embeddings [CLS]	70
10.4.4	Classification avec un classifieur léger	71
10.4.5	Évaluation	71
10.4.6	Résultats obtenues — Diogo	72
10.5	Modèle B — BERT avec fine-tuning	74
10.5.1	Principe du fine-tuning	74
10.5.2	Préparation du Dataset PyTorch	74
10.5.3	Chargement du modèle et tête de classification	75
10.5.4	Boucle d'entraînement	76

10.5.5	Évaluation	76
10.5.6	Résultats obtenues — Diogo, Ulysse, Victor et Emin	77
10.6	Modèle C — BERT spécialisé + fine-tuning	79
10.6.1	Pourquoi un modèle spécialisé?	79
10.6.2	Variantes de bases BERT disponibles	79
10.6.3	Modèles disponibles sur Hugging Face	80
10.6.4	Adaptation de la tête de classification	81
10.6.5	Fine-tuning et évaluation	81
10.6.6	Résultats obtenues — Logan	83
11	Comparaison globale — Analyse de sentiments	85
11.1	Comparaison globale des modèles	85
11.1.1	Tableau récapitulatif	85
11.1.2	Que peut-on en conclure?	86
12	Système Question-Réponse	87
12.1	Contexte et organisation	87
12.1.1	Jeux de données	89
12.1.2	Prétraitement textuel	89
12.2	Détails de l'implémentation	90
12.2.1	Indexation inversée (Lucas)	90
12.2.2	Stratégies de recherche de réponses	92
12.2.3	Évaluation des résultats	99
13	Déploiement — Application Hugging Face Spaces	101
13.1	Présentation générale	101
13.2	Architecture technique	101
13.2.1	Stack et dépendances	101
13.2.2	Chargement des modèles	102
13.2.3	Prétraitement	102
13.3	Module 1 dans l'application — Comparaison des classifieurs	102
13.3.1	Prédiction LR + TF-IDF	102
13.3.2	Prédiction BERT fine-tuné	103
13.3.3	Exemples de prédictions	103
13.4	Module 2 dans l'application — Suggestions de réponses	105
13.4.1	Construction de l'index Q-R avec cache disque	105
13.4.2	Pipeline de suggestion à la volée	106
13.4.3	Exemples de suggestions de réponses	107
13.5	Interface Gradio et accessibilité	109

IV Conclusion générale	110
14 Synthèse et analyse critique des résultats	111
14.1 Rappel des objectifs	111
14.2 Ce que nous avons appris	111
14.2.1 Les données sont le premier déterminant de la qualité	111
14.2.2 Chaque méthode de topic modeling apporte une lecture complémentaire	111
14.2.3 BERT représente un saut qualitatif significatif pour la classification	112
14.2.4 Le filtrage par sentiment BERT est le principal levier du système Q/R	113
14.3 Limites identifiées	114
15 Perspectives	115
15.1 Améliorations à court terme	115
15.2 Améliorations à moyen terme	115
15.3 Vision à long terme	115
Références	117

Première partie

Cahier des charges et organisation

Chapitre 1

Contexte et objectifs du projet

1.1 Présentation générale

La santé mentale est un enjeu majeur de la santé publique. Ce projet vise, dans un premier temps, à exploiter le jeu de données pour comprendre les types de conversations et leur structure, puis à analyser les sentiments des échanges afin de détecter les émotions et états psychologiques des patients. Ensuite, il se concentre sur l'accès et le traitement des conversations, en résumant les échanges et en identifiant les schémas linguistiques récurrents. Enfin, il prévoit le développement d'un système question-réponse, capable d'automatiser les interactions et fournir des réponses.

1.2 Ressources utilisées

- [Ensemble de données Kaggle — NLP Mental Health Conversations](#)
- [Ensemble de données Kaggle — Sentiment Analysis for Mental Health](#)
- [Page GitHub de l'équipe](#)
- [Page Hugging Face du projet](#)

Chapitre 2

Organisation et gouvernance du projet

2.1 Organisation des groupes

L'organisation a évolué en deux phases :

- **Étape 1** : Travail individuel — chaque membre traite ses données de son côté pour en extraire des informations comparables.
- **Étape 2** : Division en deux groupes distincts :
 - **Groupe 1** : Analyse des sentiments des conversations
 - **Groupe 2** : Question-réponse sur la santé mentale

2.2 Définition des rôles

La direction du projet dans sa globalité est assurée par **Logan LARROUX**, qui supervise les deux groupes et garantit la cohérence des livrables. Après constatation de l'ampleur des tâches à réaliser lors de l'étape 1, une bien meilleure organisation a été mise en place pour l'étape 2, avec des chefs de groupe dédiés à chaque axe d'analyse.

2.2.1 Organisation étape 1 : Exploration du jeu de données

Lors de l'étape 1 chaque membre de l'équipe s'est vu assigné les mêmes tâches, afin d'en comparer et de contraster les résultats.

Liste des membres :

- Ulysse Chasseigne
- Idir Yahiaoui
- Bantsoukissa Laure
- Cruz Fernandes Diogo
- Emin Belkheir
- Ezzo-Y-N Trésor PANA
- Logan Larroux
- Yvan Sellier
- Victor Blanquart-Blanchet
- Lucas Da Silva

2.2.2 Organisation étape 2 : Développement du système d'analyse de sentiments et de question-réponse

Comme dit précédemment, l'étape 2 a été divisée en deux groupes distincts, avec des rôles clairement définis pour chaque membre. En plus de cette séparation dans ses deux groupes, nous avons de nouveau séparé les tâches en sous-tâches, afin de mieux répartir le travail et d'assurer une meilleure coordination entre les membres.

Groupe 1 : Analyse des sentiments des conversations

Chef de groupe : Cruz Fernandes Diogo

Sous-groupe 1 Méthode supervisée	Sous-groupe 2 Méthode non-supervisée
Diogo	Victor
Ulysse	Emin
Logan	

Groupe 2 : Question-réponse sur la santé mentale

Chef de groupe : Yahiaoui Idir

Sous-groupe 1 Association Q&A pour chaque mot	Sous-groupe 2 Stratégie 1 Similarité Question-Question	Sous-groupe 3 Stratégie 2 Similarité Question-Réponse
Lucas	Yvan Trésor	Laure Idir

Petite précision

Les étapes de développement de chaque groupe seront détaillées plus loin dans la deuxième et troisième partie du rapport.

Deuxième partie

Étape 1 — Exploration du jeu de données

Chapitre 3

Méthodologie générale

Objectif : Comprendre la structure du corpus et identifier les thématiques majeures.

Le fichier `train.csv` comporte une grande base de données sur deux colonnes : la première est le patient conversant avec le psychologue (Context) et la seconde est la réponse du psychologue au patient (Response).

Cette première partie comporte :

- **Nettoyage des données (Pré-traitement)** : Suppression de la ponctuation, tokenisation, lemmatisation et retrait des mots vides (*stop-words*).
- **Analyse Statistique** : Création d'un tableau récapitulatif via des scripts Python (Pandas).
- **Analyse Lexicale** : Génération d'un dictionnaire lexical (n-grams) avec calcul des fréquences.
- **Modélisation et Extraction de Sujets (*Topic Modeling*)** : Quatre méthodes comparées.

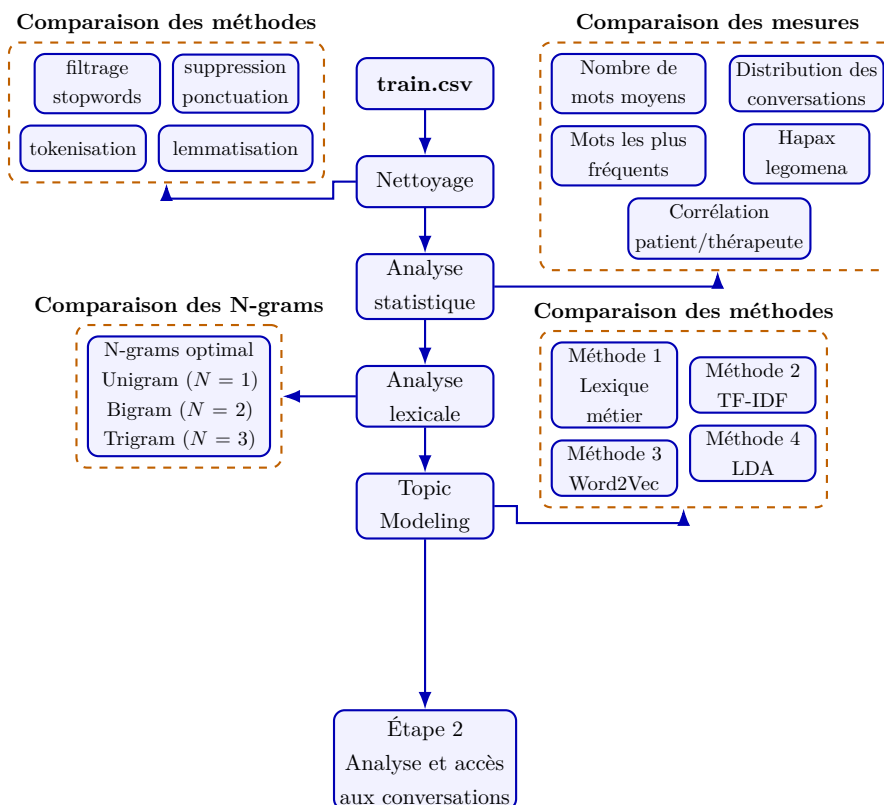


FIGURE 3.1 – Pipeline détaillé de l'étape 1 : Exploration du jeu de données

Chapitre 4

Pré-traitement des données

4.1 Nettoyage — Structure générale

Avant toute analyse, plusieurs étapes de nettoyage ont été appliquées dans l'ordre suivant.

Suppression des cellules vides et des doublons. Les lignes contenant des valeurs nulles dans les colonnes `Context` ou `Response` ont été supprimées, ainsi que les lignes entièrement dupliquées.

Filtrage des textes non-anglais. Le corpus contenait des phrases en espagnol. La bibliothèque `langdetect` a été utilisée pour détecter la langue de chaque ligne et ne conserver que les textes identifiés comme anglais. Cette détection est effectuée au niveau de la phrase entière, ce qui est bien plus fiable qu'une détection mot par mot.

```
from langdetect import detect, LangDetectException

def is_english_text(text):
    try:
        return detect(str(text)) == 'en'
    except LangDetectException:
        return True # texte trop court -> on garde

mask = df['Context'].apply(is_english_text) \
        & df['Response'].apply(is_english_text)
df = df[mask].reset_index(drop=True)
```

Normalisation. Tous les textes ont été passés en minuscules afin d'éviter que *"Toto"* et *"toto"* soient traités comme deux mots distincts. Les espaces insécables (`\xa0`), fréquents dans des données extraites du web, ont également été supprimés.

4.1.1 Tokenisation

La tokenisation consiste à découper chaque phrase en unités élémentaires appelées *tokens*. La bibliothèque `spacy` (`en_core_web_sm`) a été utilisée pour cette étape, via `nlp.pipe()` afin de traiter les données en batch et réduire le temps de traitement.

4.1.2 Suppression de la ponctuation et des mots vides

Les *stopwords* (mots vides) sont des mots sans réelle signification sémantique (pronoms, ad-
verbes, mots de liaison). spaCy les identifie automatiquement via l'attribut `token.is_stop`.
Les tokens de ponctuation ont été supprimés de la même façon avec `token.is_punct`.

Des mots trop génériques mais non reconnus comme stopwords ont également été filtrés
manuellement, voici deux exemple de listes d'`extra_stopwords` utilisées par Logan et
Lucas :

```
# Exemple - Logan
extra_stopwords = {
    "feel", "like", "want", "know", "time",
    "thing", "good", "need", "year", "think",
    "tell", "say", "come", "go"
}
```

```
# Exemple - Lucas
mots_inutiles = set([
    "i", "me", "my", "myself", "we", "our", "ours", "ourselves",
    # ... [liste complete dans le notebook]
    "there", "back", "even", "every", "years", "told", "going", "
    still"
])
```

4.1.3 Lemmatisation

La lemmatisation transforme chaque mot en sa forme de base ("*running*" → "*run*", "*better*"
→ "*good*"). Cette étape réduit drastiquement le nombre de tokens distincts et regroupe les
formes similaires. Elle permet également de filtrer les URLs, emails et chiffres dans une
seule passe :

```
df["Context_clean"] = [
    [
        token.lemma_
        for token in doc
        if not token.is_stop
        #...et autres filtres (is_punct, is_url, is_email, etc.)
        and token.lemma_ not in extra_stopwords
    ]
    for doc in docs_context
]
```

Remarque importante

`spaCy` lemmatise différemment selon le contexte grammatical. Par exemple, *"feeling"* utilisé comme nom reste tel quel au lieu d'être ramené à *"feel"*. C'est pourquoi *"feel"* a été retiré manuellement des `extra_stopwords` pour éviter les doublons sémantiques.

4.2 Nettoyage — Approches comparées

Puisque ce pipeline n'a pas été imposé, chaque membre a pu appliquer son propre pré-traitement, avec des niveaux de filtrage différents. Prenons l'exemple de deux approches contrastées, celle d'Emin et celle de Diogo.

4.2.1 Approche minimaliste (Emin)

Celle-ci est la plus simple, elle se limite à la suppression des lignes vides et au reset de l'index. Le résultat est un nombre de patients et de thérapeutes plus élevé, mais avec un risque de bruit plus important dans les données.

```
# Suppression des lignes vides
df = df.dropna(how='all')
df = df.dropna(subset=['Context'])
df = df.dropna(subset=['Response'])
df = df.reset_index(drop=True)
```

4.2.2 Approche étendue (Diogo)

Une version beaucoup plus étendue est celle de Diogo, qui inclut un nettoyage plus poussé du texte, ainsi qu'un filtrage pour ne garder que les conversations en anglais.

Cette approche réduit le nombre de patients et de thérapeutes uniques, mais améliore la qualité des données en éliminant les réponses non pertinentes ou dans une langue différente.

Son pipeline est donc le suivant :

- Nettoyage du texte : suppression de la ponctuation, des caractères spéciaux, des espaces multiples, et conversion en minuscules.
- Filtrage linguistique : utilisation de la bibliothèque `langdetect` pour ne conserver que les conversations en anglais.
- Suppression des stop-words : retrait des mots vides courants en anglais pour se concentrer sur les termes significatifs.
- Suppression des doublons : élimination des conversations identiques pour éviter les biais dans les analyses statistiques.

- Réinitialisation de l'index : après les filtrages, l'index est réinitialisé pour une meilleure gestion des données.

Lemmatisation et tokenisation

L'application de la lemmatisation (et de la tokenisation) pour l'analyse du texte a été un débat au sein de l'équipe. La grande majorité de l'équipe a décidé de ne pas l'appliquer lors de l'analyse statistique et lexicale du corpus, afin de conserver les formes originales des mots.

4.3 Résultats

4.3.1 Nombre de patients uniques

Résultat	Nb. Membres	Nom Membres	Méthode
995	8 personnes	Ulysse - Emin - Victor - Laure - Trésor - Yvan - Idir - Lucas	Sans prétraitement
819	1 personne	Logan	Prétraitement + filtre espagnol
796	1 personne	Diogo	Prétraitement + filtre espagnol

4.3.2 Nombre de thérapeutes uniques

Résultat	Nb. Membres	Nom Membres	Méthode
2 479	6 personnes	Emin - Victor - Idir - Laure - Yvan - Lucas	Sans prétraitement
2 472	1 personne	Ulysse	Léger prétraitement
2 409	1 personne	Logan	Prétraitement complet
≥ 12	1 personne	Diogo	Pattern matching

Limite importante

Les valeurs autour de 2 400–2 479 correspondent très probablement au nombre de lignes/réponses dans le fichier, et non au nombre réel de thérapeutes distincts. En l'absence d'identifiant unique (ID), il est impossible de différencier les thérapeutes entre eux avec certitude.

4.3.3 Nombre de mots moyen par patient / thérapeute

Côté patients		Côté thérapeutes	
Membre	Mots moyens	Membre	Mots moyens
Victor	66,86	Emin	177,2
Emin	55,2	Victor	140,2
Idir	24,8	Idir	84,8
Lucas	21,3	Lucas	82,4
Logan	16,3	Logan	60,1

Interprétation

On observe un rapport allant du simple au quadruple selon le niveau de filtrage. Dans chaque cas, **les thérapeutes ont tendance à produire des réponses plus longues que les patients**, ce qui est cohérent avec la nature de leur rôle dans la conversation.

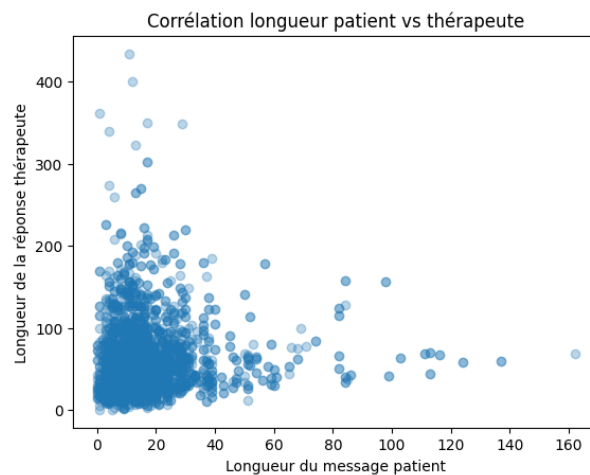


FIGURE 4.1 – Corrélation entre le nombre de mots côté patient et côté thérapeute — Logan

Maintenant que nous avons une idée de la structure générale du corpus, nous allons pouvoir nous pencher plus en détail sur les **thématiques abordées dans les conversations**, à travers une analyse lexicale afin de mieux comprendre les sujets centraux des échanges.

Chapitre 5

Analyse lexicale et statistique

5.1 Mots les plus fréquents

La fréquence des mots les plus utilisés dans les conversations peut révéler les thématiques centrales abordées par les patients et les thérapeutes. Cependant, **il est crucial de filtrer les mots vides** (stop-words) pour éviter que des termes génériques ne dominent l'analyse.

5.1.1 Sans filtrage renforcé

Ulysse, Emin, Victor, Idir, Trésor et Yvan ont appliqué un filtrage minimal, ce qui a permis de conserver des mots très fréquents mais peu informatifs.

Côté patients	Côté thérapeutes	Global
<i>im, feel, dont</i>	<i>feel, may, help, like</i>	<i>feel, like, may</i>

5.1.2 Avec filtrage renforcé

Diogo, Logan, Laure et Lucas eux ont appliqué un filtrage plus poussé, en retirant des mots génériques comme *feel, like, may*, qui sont très fréquents mais peu spécifiques.

Côté patients	Côté thérapeutes	Global
<i>relationship, love, anxiety</i>	<i>relationship, like</i>	<i>relationship, help</i>

5.1.3 Interprétation

- **Sans filtrage renforcé** → expressions émotionnelles brutes des patients (*feel, love*) et posture d'aide des thérapeutes (*may, help, would*).
- **Avec filtrage renforcé** → thématiques explicites : *relationship, anxiety, love* côté patient ; *relationship* côté thérapeute.

Remarque

Ces résultats confirment que le corpus est centré sur les relations interpersonnelles, sujet le plus fréquent, couplé à des états psychologiques comme l'anxiété.

5.2 Analyse lexicale — N-grams

5.2.1 Définition

Un **n-gram** est une séquence de n mots consécutifs dans un texte. Cette approche permet de capturer non seulement les mots isolés, mais aussi les associations de mots fréquentes.

- **Unigram (n=1)** : mots individuels, ex. "*anxiety*", "*depression*".
- **Bigram (n=2)** : paires de mots, ex. "*mental health*", "*feel like*".
- **Trigram (n=3)** : triplets de mots, ex. "*feel like cry*", "*family support system*".
- ainsi de suite...

Dans notre travail, nous allons devoir repérer des formulations récurrentes qui ont du sens :

Type	Exemple	Interprétation
Unigram (n=1)	<i>anxiety</i> (fréq. 342)	Thème dominant
Bigram (n=2)	<i>mental health</i> (fréq. 87)	Expression caractéristique
Trigram (n=3)	<i>feel like cry</i> (fréq. 54)	Patron de détresse

TABLE 5.1 – Exemples de n-grams et leur interprétation

5.2.2 Choix du n optimal

Il n'existe pas de valeur universelle pour n . Deux indicateurs ont été utilisés pour guider ce choix :

- **Fréquence moyenne** : si elle chute sous un seuil de 5, les n-grams sont trop rares pour être statistiquement significatifs.
- **Pourcentage d'hapax** : si plus de 70 % des n-grams n'apparaissent qu'une seule fois, la taille n est trop grande pour le corpus.

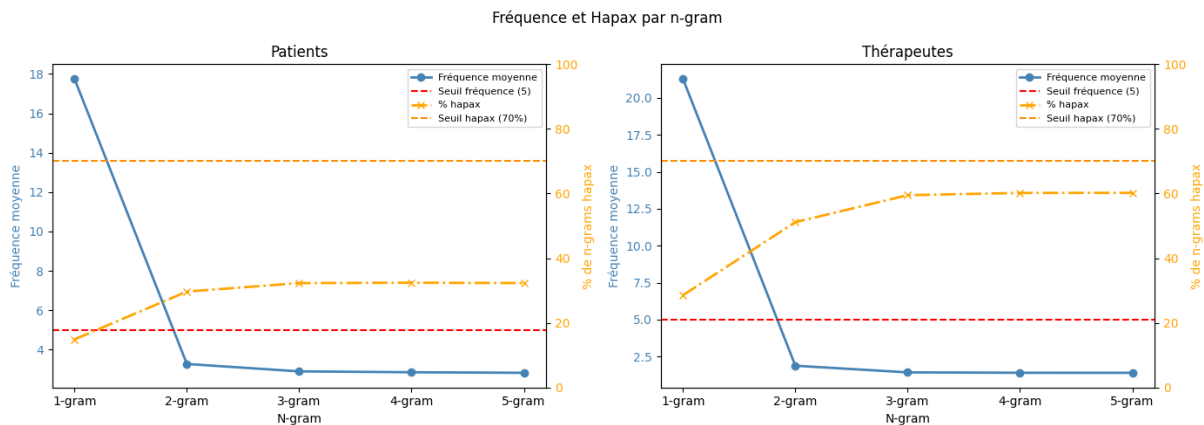


FIGURE 5.1 – Graphe de fréquence moyenne et pourcentage d'hapax en fonction de n

On remarque qu'à partir de $n = 1$, la fréquence diminue déjà fortement. Cependant, les bigrams et trigrams restent exploitables car le seuil d'hapax n'est pas dépassé.

5.2.3 Résultats — Logan, Victor et Lucas

Les bigrams et trigrams apportent un contexte nettement plus riche que les unigrams seuls. Ils peuvent permettre par la suite d'identifier des patrons linguistiques et des thèmes récurrents pour le topic modeling mais malheureusement leur fréquence est relativement faible, ce qui rend leur exploitation plus délicate. . .

Unigram	fréquence	Unigram	fréquence
<i>relationship</i>	486	<i>help</i>	2150
<i>love</i>	431	<i>relationship</i>	1933
<i>friend</i>	369	<i>way</i>	1668

TABLE 5.2 – Trois plus fréquent **unigram** pour les patients (gauche) et thérapeutes (droite)

Bigram	fréquence	Bigram	fréquence
<i>issue address</i>	94	<i>therapist help</i>	160
<i>self esteem</i>	65	<i>ask question</i>	154
<i>month ago</i>	64	<i>mental health</i>	153

TABLE 5.3 – Trois plus fréquent **bigram** pour les patients (gauche) et thérapeutes (droite)

Trigram	fréquence	Trigram	fréquence
<i>history sexual abuse</i>	51	<i>hello thank question</i>	68
<i>low self esteem</i>	50	<i>mental health professional</i>	52
<i>couple therapy session</i>	48	<i>landwehr dbh lpc</i>	51

TABLE 5.4 – Trois plus fréquent **trigram** pour les patients (gauche) et thérapeutes (droite)

Maintenant que nous avons une idée des mots les plus fréquents, nous allons devoir identifier les thématiques majeures abordées dans les conversations. Pour cela, nous allons appliquer différentes méthodes de **topic modeling**, en comparant leurs avantages et limites respectifs.

Chapitre 6

Topic Modeling

6.1 Préambule

Le **topic modeling** est une technique d'extraction de thèmes latents à partir d'un corpus de textes. Il existe plusieurs méthodes pour réaliser cette tâche, chacune avec ses propres avantages et limites. Dans cette section, nous allons présenter quatre approches différentes pour identifier les thématiques majeures dans les conversations de santé mentale, en commençant par une méthode manuelle basée sur un lexique personnalisé, puis en explorant des méthodes plus automatisées et robustes comme TF-IDF, Word2Vec et LDA.

Remarque importante

Chaque membre de l'équipe a appliqué sa propre version de ces méthodes, ce qui nous a permis d'avoir une diversité d'approches et de résultats à comparer. Par question de clareté, nous allons présenter les résultats de chaque méthode de manière synthétique, en présentant les résultats les plus pertinents et en discutant de leurs avantages et limites respectifs.
(voir les notebooks¹ individuels pour les détails de chaque implémentation).

Méthode	Membres
Lexique personnalisé	Diogo, Trésor, Emin, Logan, Victor, Idir, Trésor, Yvan, Lucas
TF-IDF + K-Means	Ulysse, Diogo, Trésor, Emin, Logan, Victor, Idir, Laure, Trésor, Yvan, Lucas
Word2Vec + Clustering	Ulysse, Diogo, Trésor, Emin, Logan, Victor, Idir, Trésor, Yvan, Lucas
LDA (Latent Dirichlet Allocation)	Ulysse, Diogo, Trésor, Emin, Logan, Victor, Idir, Trésor, Yvan, Lucas

TABLE 6.1 – Répartition des méthodes de topic modeling entre les membres de l'équipe

1. [Liens github des notebooks](#)

6.2 Méthode 1 : Lexique personnalisé

6.2.1 Principe

Un dictionnaire thématique est construit **manuellement**, en associant chaque thème de santé mentale à une liste de mots-clés. Chaque question peut être assignée à plusieurs thèmes simultanément (classification multi-label).

Le but de cette première section était de nous introduire le principe de dictionnaire thématique, ainsi que de nous familiariser avec les thématiques récurrentes dans les conversations de santé mentale.

Remarque

Ce filtrage manuel étant bien entendu subjectif, il ne sera pas appliqué dans les méthodes de topic modeling, qui sont plus robustes à ce type de bruit lexical.

Nous allons présenter deux exemples de lexiques, l'un minimaliste (Trésor) et l'autre plus étendu (Diogo). Bien entendu chaque membre de l'équipe a réalisé son propre lexique.

6.2.2 Approche minimaliste (Trésor)

```
lexique = {
    'Anxiete':    ['anxious', 'anxiety', 'panic', 'fear'],
    'Depression': ['depress', 'depression', 'sad', 'suicide'],
    'Relation':   ['relationship', 'husband', 'wife', 'partner']
}
```

6.2.3 Approche étendue (Diogo)

```
LEXICON = {
    'Anxiety':    ['anxiety', 'anxious', 'panic', 'worry', '
        worried', 'nervous',
        'fear', 'phobia', 'stress', 'stressed', '
        overthink', 'dread'],
    # 11 autres thèmes contenant chacun 12 mots...
}
```


6.2.4 Résultats

Les thèmes apparaissant de façon transversale :

- **Anxiété** (*anxiety, panic, stress*)
- **Dépression** (*depression, sad, hopeless*)
- **Relations** (*relationship, love, partner*) — thème le plus fréquent

6.2.5 Évaluation critique

Avantage	Limite
Très interprétable et explicable	Rigide : ne capte que les mots explicitement listés
Facile à adapter au domaine	Ne gère pas les synonymes non prévus
Résultats stables et reproductibles	Nécessite une expertise métier
Classification multi-label naturelle	Insensible au contexte d'usage d'un mot

Toute l'équipe est en accord avec trois thématiques principales : anxiété, dépression et relations. Or cela ne signifie pas que ses thématiques sont les plus fréquentes, bien au contraire ! D'autres thèmes bien plus spécifique auxquels on ne penserait pas peuvent faire leur apparition. D'où l'intérêt de méthodes plus robustes et moins sujetives que le lexique manuel.

6.3 Méthode 2 : TF-IDF + K-Means

6.3.1 Principe de TF-IDF

Le **TF-IDF** (*Term Frequency — Inverse Document Frequency*) est une méthode de pondération évaluant l'importance d'un mot au sein d'un document par rapport à l'ensemble du corpus. Il combine deux composantes :

- **Term Frequency (TF)** : fréquence d'un terme dans le document.

$$tf_{i,j} = \frac{n_{i,j}}{\sum_k n_{k,j}}$$

où $n_{i,j}$ est le nombre d'occurrences du terme i dans le document j .

- **Inverse Document Frequency (IDF)** : pondération par la rareté du mot dans le corpus.

$$idf(w) = \log\left(\frac{N}{df_t}\right)$$

où N est le nombre total de documents et df_t le nombre de documents contenant le terme t .

Le score final est :

$$w_{i,j} = tf_{i,j} \times \log\left(\frac{N}{df_i}\right)$$

Les vecteurs sont ensuite normalisés par la norme L2 afin que la longueur des documents ne biaise pas les comparaisons.

Grâce au pré-traitement déjà effectué (suppression des stopwords, lemmatisation), l'IDF discrimine mieux les termes et les scores sont plus significatifs.

Vectorisation TF-IDF

On ne peut malheureusement pas utiliser TF-IDF directement sur les textes bruts, il faut d'abord les vectoriser. Vectoriser un texte signifie le transformer en une représentation numérique, pour que par la suite les algorithmes puissent les traiter.

Pour cela, nous allons utiliser la fonction `TfidfVectorizer` de la bibliothèque **scikit-learn**. Cette fonction a de grands avantages, elle permet de :

- lister tous les mots uniques
- définir le score TF
- définir le score IDF
- définir le score final et vecteur L2

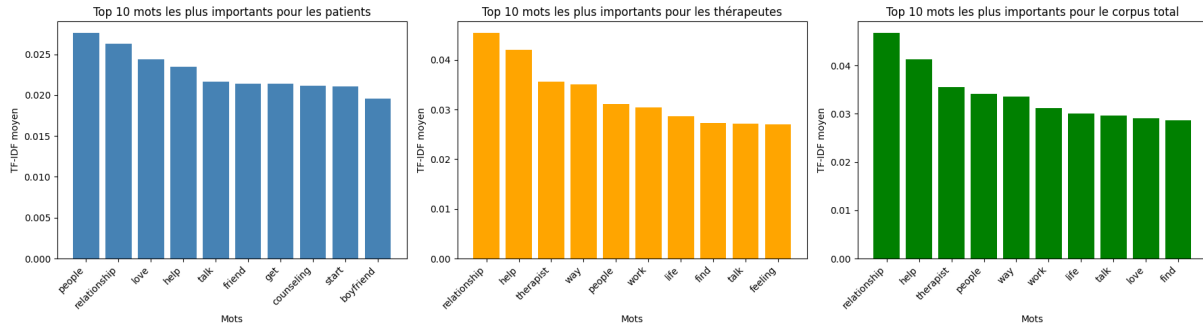


FIGURE 6.1 – Exemple d’histogramme des 10 mots les plus importants selon le score TF-IDF (Logan)

Clustering K-Means

Après avoir vectorisé les textes en utilisant TF-IDF, nous obtenons une matrice de taille $N \times M$ (où N est le nombre de documents et M le nombre de mots uniques). Grâce à cette matrice, nous allons pouvoir maintenant regrouper les mots similaires en **clusters**, or nous devons dans un premier temps définir le nombre de clusters K à utiliser. Un des risques est de choisir un K trop petit ou trop grand, ce qui peut conduire à des clusters peu significatifs ou à un sur-apprentissage.

6.3.2 Sélection du nombre de clusters (K)

Plusieurs stratégies explorées :

- Définition manuelle
- Méthode du coude (*elbow method*)
- Équilibrage des tailles

Remarque

Une méthode que nous aurions pu explorer mais que nous n’avons pas fait est la **silhouette score**, qui mesure la cohésion et la séparation des clusters pour différents K . Combiné avec la méthode du coude, cela aurait pu nous aider à trouver un compromis entre la qualité des clusters et leur interprétabilité.

6.3.3 Résultats selon K

$K = 6$ — Clusters déséquilibrés — Logan

La méthode du coude (*elbow method*) a été utilisée pour déterminer le nombre optimal de clusters K . L'inertie diminue fortement jusqu'à $K = 6$, puis la courbe se stabilise.

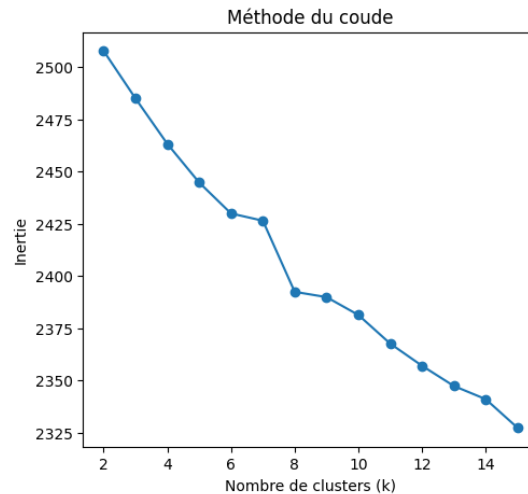


FIGURE 6.2 – Graphe de la méthode du coude pour déterminer le nombre optimal de clusters K

$K = 6$ a donc été retenu.

Thème identifié	Effectif
Dépression sévère — idées suicidaires	457
Mal-être général — perte de motivation	112
Dépression sévère — fatigue mentale	46
Thérapie de couple — anxiété	223
Traumatisme — abus — maladie grave	1 718
Problèmes familiaux — divorce — identité	139

Résultat peu satisfaisant

Fort déséquilibre de taille (46 vs 1 718), thèmes qui se recoupent. Une des solutions aurait été d'agrandir la plage de K testée, mais cela aurait conduit à des clusters encore plus petits et moins interprétables.

$K = 3$ — Meilleure configuration avec LSA — Diogo**Très petite explication du LSA**

La **LSA** (*Latent Semantic Analysis*) est une technique de réduction de dimension appliquée à la matrice TF-IDF. Elle repose sur la **décomposition en valeurs singulières** (SVD) :

$$M = U \cdot \Sigma \cdot V^\top$$

où $M \in \mathbb{R}^{d \times n}$ est la matrice terme-document (d termes, n documents), U les vecteurs de termes, Σ la matrice diagonale des valeurs singulières, et $V^\top$ les vecteurs de documents dans l'espace latent.

En ne conservant que les k premières composantes ($k \ll d$), on projette les documents dans un espace sémantique réduit avec deux avantages principaux :

- **Réduction du bruit lexical** : des termes synonymes (ex. *sad* et *depressed*) qui co-occurrent dans des contextes similaires se rapprochent dans l'espace réduit.
- **Amélioration du clustering** : K-Means est sensible à la *malédiction de la dimensionnalité* ; réduire la dimension améliore la qualité des clusters.

Dans l'implémentation de Diogo, la LSA est appliquée via `TruncatedSVD` de `scikit-learn` avec 50 composantes, après vectorisation TF-IDF :

```
from sklearn.decomposition import TruncatedSVD
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import Normalizer

svd = TruncatedSVD(n_components=50, random_state=42)
X_lsa = make_pipeline(X)
X_norm = Normalizer().fit_transform(X_lsa) # Normalisation L2
```

La normalisation finale (norme L2) garantit que les distances cosinus restent interprétables après la projection. Comme montré dans les résultats ($K = 3$), l'ajout de la LSA conduit à des clusters plus équilibrés et thématiquement plus distincts par rapport au clustering TF-IDF seul.

Cela nous permet d'obtenir des clusters plus équilibrés et thématiquement plus distincts, couvrant 100 % du corpus.

Topic	Effectif	Part	Mots-clés principaux
Family and relational dynamics	309	38,8 %	friends, family, child, dad, mom, husband
Emotional distress and mental health	284	35,7 %	anxiety, depression, stop, start, school
Romantic relationships and intimacy	203	25,5 %	love, relationship, boyfriend, sex, girl

Même si les résultats sont meilleurs, on observe que les thèmes restent assez larges et se recoupent partiellement (ex. *family* peut apparaître dans les deux premiers clusters).

6.3.4 Évaluation critique

Bien que la méthode TF-IDF + K-Means soit rapide et facile à implémenter, elle présente plusieurs limites :

Avantage	Limite
Méthode rapide et scalable	Sensible au choix de K
Bien documentée et reproductible	Ignore la sémantique contextuelle des mots
Compatible avec LSA pour de meilleurs résultats	Mots traités comme des entités indépendantes

Ses limites peuvent être partiellement atténuées en combinant avec une réduction de dimension (LSA) ou en utilisant des représentations plus riches comme les word embeddings, **ce qui nous amène à la méthode suivante.**

6.4 Méthode 3 : Word Embeddings

6.4.1 Principe du Word Embedding

Le **Word Embedding** (plongement lexical) est une représentation sémantique des mots sous forme de vecteurs de nombres réels. Deux mots de sens proche ont des vecteurs proches dans l'espace mathématique.

Contrairement à TF-IDF qui est une **représentation statistique** (fréquence d'apparition), Word2Vec est une **représentation sémantique** : il apprend ce que le mot signifie en observant les mots qui l'entourent dans le corpus. Ainsi, "*sad*" et "*depressed*" se trouvent proches dans l'espace vectoriel parce qu'ils apparaissent dans des contextes similaires.

6.4.2 Skip-Gram vs CBOW

Word2Vec propose deux architectures :

- **CBOW** (*Continuous Bag of Words*) : prédit le mot central à partir de son contexte. Très rapide sur les grands corpus, mais la moyenne du contexte *lisse le bruit*, ce qui peut effacer des nuances importantes.
- **Skip-Gram** (*sg=1*) : prédit le contexte à partir du mot central. Plus adapté aux **corpus petits et spécialisés** car il génère plus d'exemples d'entraînement par mot, compensant le manque de données.

Remarque

Étant donné que notre corpus est relativement **petit et spécialisé**, le mode **Skip-Gram** est **généralement recommandé**^a pour capturer des relations sémantiques plus fines.

^a. Référence à cette décision : [geeksforgeeks.org](https://www.geeksforgeeks.org)

6.4.3 Word2Vec (Skip-gram) — Logan

La version de logan implémente Word2Vec en mode Skip-Gram, avec les hyperparamètres suivants :

- **vector_size=100** : chaque mot est représenté par un vecteur de 100 dimensions, ce qui permet de capturer une sémantique riche tout en restant gérable pour le clustering.
- **window=5** : le contexte de chaque mot est défini comme les 5 mots précédents et les 5 mots suivants, ce qui permet de capturer des relations à la fois locales et globales.
- **min_count=3** : les mots apparaissant moins de 3 fois sont ignorés, ce qui réduit le bruit et améliore la qualité des embeddings.
- **epochs=10** : le modèle est entraîné pendant 10 itérations sur le corpus, ce qui permet d’obtenir des vecteurs stables sans sur-apprentissage.
- **seed=42** : pour assurer la reproductibilité des résultats.
- **sg=1** : pour activer le mode Skip-Gram, qui est plus adapté à notre corpus de taille modérée.

Résultats avec $K = 6$ (patients)

Cluster	Thème	Effectif
0	Dépression	893
1	Problèmes sexuels — santé physique	21
2	Thérapie de couple — anxiété	414
3	Mal-être général	1 084
4	Anxiété — trouble du sommeil	195
5	Traumatisme	88

Le problème que l’on pourrait avoir sur cette méthode est que les clusters sont très déséquilibrés, ce qui rend l’interprétation plus difficile. La prochaine méthode que nous allons présenter, basée sur les **Sentence Transformers**, permet de mieux équilibrer les clusters et d’obtenir des thématiques plus distinctes.

6.4.4 Sentence Transformer (BERT / all-MiniLM-L6-v2) — Idir

Introduction rapide à BERT

BERT (*Bidirectional Encoder Representations from Transformers*) est un modèle de langage pré-entraîné développé par Google en 2018. Contrairement à Word2Vec, qui génère des embeddings statiques pour chaque mot, BERT produit des embeddings contextuels : le vecteur d'un mot dépend donc de la phrase dans laquelle il apparaît.

Remarque

Le modèle BERT, ayant sa partie dédiée dans la troisième partie, sera expliqué bien plus en détails à ce moment là.

Résultats avec $K = 8$

Effectif	Thème du cluster	Mots représentatifs
111	Marriage and infidelity	husband, married, relationship, love, wife
86	Seeking therapy and abuse	therapist, counseling, child, therapy, issues
121	Romantic relationships and dating	love, relationship, boyfriend, guy, dating
101	Family and parenting	mom, dad, child, mother, family
95	Anxiety and mental disorder	anxiety, depression, attacks, sleep, panic
105	Social anxiety and communication	thoughts, friends, talking, upset, anger
75	Sexuality and gender identity	sex, girl, love, men, afraid, girls
102	Depression and social life	friends, school, sad, depressed, family

Remarque

Meilleur résultat de la méthode 3 : clusters de taille homogène (≈ 75 – 121), thèmes distincts et bien interprétables.

Les clusters obtenus avec les embeddings de BERT sont plus équilibrés et thématiquement plus distincts que ceux obtenus avec Word2Vec, ce qui confirme l'importance de la contextualisation dans la représentation des textes. Une autre façon est d'obtenir des **embeddings de phrase** est d'utiliser les **embeddings moyennés de spaCy**, mais cela conduit à des résultats moins satisfaisants, comme nous allons le voir dans la section suivante.

6.4.5 spaCy (embeddings moyennés) — Emin

Dans cette approche, les embeddings de chaque mot sont extraits à l'aide de spaCy, puis moyennés pour obtenir une représentation vectorielle de chaque document. Cependant, cette méthode présente plusieurs limites :

- **Perte de contexte** : en moyennant les embeddings de tous les mots, on perd la structure syntaxique et les relations entre les mots, ce qui peut conduire à des représentations moins précises.
- **Mots peu informatifs** : les mots fréquents et peu discriminants (ex. *know*, *like*, *feel*) peuvent dominer la représentation, masquant les thèmes plus spécifiques.
- **Résultats moins cohérents** : les clusters obtenus sont souvent moins thématiquement distincts et plus difficiles à interpréter que ceux obtenus avec des modèles plus avancés comme BERT.

Résultats avec $K = 5$

Cluster	Mots principaux
0	counseling, therapist, know, end, client, counselor
1	dont, im, like, feel, know, want
2	counseling, really, anything, help, people, something
3	im, ive, like, feel, get, many
4	feel, years, time, im, like, relationship

Résultat insatisfaisant

Clusters 0 et 2 très similaires, plusieurs clusters dominés par des mots peu informatifs.

6.4.6 Bilan comparatif — Méthode 3

Modèle	Qualité	Points forts	Limites
Sentence Transformer	Très bonne	Contexte global, tailles homogènes	Modèle lourd
Word2Vec (Skip-gram)	Bonne	Granularité thématique fine	Déséquilibre de taille
spaCy	Passable	Rapide, simple	Perte de contexte

L'équipe entière s'accorde pour dire que les embeddings de BERT (*via Sentence Transformers*) offrent une meilleure qualité de clustering que les embeddings moyennés de spaCy ou les vecteurs Word2Vec, grâce à leur capacité à capturer le contexte et les nuances

sémantiques des textes. Cependant, cette méthode est **plus lourde à entraîner et à utiliser**, ce qui peut être un frein pour des applications en temps réel ou sur des ressources limitées. La dernière méthode que nous allons présenter, basée sur **LDA**, offre une approche différente en se concentrant sur la **modélisation thématique** plutôt que sur la représentation sémantique.

6.5 Méthode 4 : LDA (Latent Dirichlet Allocation)

6.5.1 Qu'est ce que le LDA ?

Le **LDA** (*Latent Dirichlet Allocation*) est un modèle génératif probabiliste permettant de découvrir des sujets latents dans un corpus. C'est une approche bayésienne qui suppose que chaque document est un **mélange de thèmes**, et que chaque thème est une **distribution de mots**.

Contrairement à K-Means où chaque document appartient à un unique cluster, LDA assigne à chaque conversation un mélange de thèmes avec des probabilités. Par exemple :

"I feel sad and anxious" $\rightarrow$ 70 % dépression + 30 % anxiété

LDA produit deux sorties :

1. **Thèmes** $\rightarrow$ **mots** : distribution des mots caractéristiques pour chaque thème.
2. **Documents** $\rightarrow$ **thèmes** : proportion de chaque thème dans chaque conversation.

Dans cette méthode, nous allons utiliser la bibliothèque **gensim** pour implémenter LDA, en utilisant les représentations de documents au format bag-of-words (corpus) et le dictionnaire de mots (id2word). De plus, selon les membres, nous avons défini un nombre de thèmes *N_TOPICS* et un nombre de passes *PASSES* pour l'entraînement du modèle.

```
N_TOPICS = 6
PASSES = 15

model = LdaModel(
    corpus=corpus,
    id2word=dictionary,
    num_topics=n_topics,
    random_state=42,
    passes=passes
)
```

FIGURE 6.3 – Exemple d'implémentation de LDA avec la bibliothèque gensim — Logan

6.5.2 Résultats à 6 topics — Logan

Thème	Mots représentatifs
Relations de couple et infidélité	relationship, husband, child, listen, cheat, woman
Relations familiales et soutien social	relationship, people, person, help, dad, family
Relations amoureuses et communication	love, friend, talk, boyfriend, right, therapist
Anxiété et gestion émotionnelle	anxiety, help, talk, friend, feeling, get, try
Détresse émotionnelle et conflits	thought, boyfriend, girl, life, have, stop, cry
Thérapie et problématiques de couple	normal, issue, wife, therapy, walk, couple, self

Interprétation

Sur 6 topics, on retrouve des thèmes cohérents et distincts, couvrant les relations de couple, les relations familiales, l'anxiété, la détresse émotionnelle et la thérapie. Cependant, certains thèmes se recoupent partiellement (ex. *relationship* apparaît dans plusieurs topics), ce qui est une limite de l'approche.

6.5.3 Résultats à 6 topics — Diogo

Thème	Mots représentatifs
Romantic and sexual issues	love, hes, sex, relationship, boyfriend
Mental health and disorders	anxiety, depression, shes, boyfriend, mom
Therapy, communication and coping	school, friends, right, high, live
Relationship trust and family	girlfriend, thinking, hes, thoughts, stop
Intrusive thoughts and past	mother, mom, dad, son, child
Family anger and conflict	therapist, money, phone, relationship, friends

Interprétation

Pour Diogo, les thèmes sont similaires à ceux de Logan, mais avec des nuances différentes. Par exemple, le thème "*Therapy, communication and coping*" regroupe des mots liés à la vie quotidienne et à la gestion de l'anxiété, tandis que "*Relationship trust and family*" met davantage l'accent sur les relations de confiance et les dynamiques familiales.

6.5.4 Résultats à 5 topics — Lucas

Thème	Mots représentatifs
Santé mentale, sexualité et famille	sexual, family, self, depression, married, anxiety
Relations amoureuses et questionnements	say, right, boyfriend, think, girlfriend, sex, love
Anxiété, vie quotidienne et thérapie	think, boyfriend, couple, school, anxiety, therapy
Détresse émotionnelle et pensées nég.	bad, think, sex, relationship, afraid, stop, love
Relations de couple et vie familiale	boyfriend, make, counseling, child, life, talk

Interprétation

Avec 5 topics, les thèmes sont plus larges et moins distincts que dans les configurations à 6 topics. Par exemple, le thème "*Santé mentale, sexualité et famille*" regroupe des concepts très différents, ce qui rend l'interprétation plus difficile.

6.5.5 Évaluation critique

Lors de l'analyse des ses représentations thématiques, nous avons identifié plusieurs avantages et limites de la méthode LDA :

Avantage	Limite
Topics interprétables et cohérents	Choix du nombre de topics difficile
Assignation douce = nuance inter-topics	Sensible au prétraitement
Bien adapté aux textes courts	Résultats variables selon hyperparamètres
Fondement probabiliste rigoureux	Ne capture pas le contexte long

Les résultats obtenues avec LDA sont généralement plus interprétables que ceux obtenus avec les méthodes basées sur les embeddings, car ils fournissent une distribution de mots pour chaque thème. Cependant, le choix du nombre de topics (N_TOPICS) est crucial et peut grandement influencer la qualité des résultats. De plus, **LDA ne capture pas les relations à long terme entre les mots**, ce qui peut limiter sa capacité à identifier des thèmes plus complexes.

Mais alors dans ce cas, quelle méthode est la meilleure ? Nous allons faire une conclusion comparée de toutes les méthodes présentées dans ce chapitre.

6.6 Comparaison des méthodes

Méthode	Forces	Faiblesses	Usage recommandé
Lexique manuel	Interprétable, contrôlable	Rigide, non-généralisant	Catégorisation ciblée
TF-IDF + K-Means	Rapide, scalable	Ignore le contexte	Exploration initiale
Word2Vec	Similarités sémantiques	Déséquilibre possible	Représentation de mots
Sentence Transformer	Contexte global, homogène	Lourd, moins interprétable	Meilleur clustering
LDA	Assignation douce	Chevauchement de topics	Analyse thématique

Recommandation globale

- Pour **interpréter** les données : **LDA** est le plus lisible et le plus informativement riche.
- Pour **regrouper** les documents : **Sentence Transformer** (BERT) offre la meilleure qualité de clustering.
- Pour une **catégorisation stricte** : le **lexique manuel** reste le plus fiable si le domaine est bien défini.

Que peut-on en conclure ? Il n'existe pas de méthode universelle pour l'analyse de texte, et le choix dépend fortement des objectifs spécifiques, de la nature du corpus et des ressources disponibles. Dans notre cas spécifique, la combinaison de plusieurs méthodes (ex. LDA pour l'analyse thématique + Sentence Transformer pour le clustering) pourrait offrir une approche plus complète et nuancée. Cependant, cela nécessite une expertise plus approfondie et une validation rigoureuse pour éviter les biais et les interprétations erronées.

Chapitre 7

Bilan de l'étape 1

7.1 Problèmes techniques identifiés

Lors de l'étape 1, plusieurs problèmes techniques ont été identifiés qui ont impacté la qualité et la comparabilité des résultats entre les membres du groupe.

Problème	Description	Impact
Hétérogénéité du nettoyage	Chaque membre a appliqué un pipeline différent	Résultats non directement comparables
Gestion des doublons	Nombre de patients variant de 796 à 995	Biais dans toutes les statistiques
Commentaires insuffisants	Manque de documentation dans les notebooks	Difficultés à comprendre le code des autres
Choix du nombre de clusters	Méthode du coude peu discriminante après $K = 6$	Difficulté à trouver un K optimal
Normalisation des sorties	Formats de résultats différents entre membres	Comparaison manuelle laborieuse

Ses résultats ont conduit à une revue complète de l'organisation pour la prochaine étape, avec une standardisation des résultats et une meilleure documentation pour faciliter la collaboration.

7.2 Conclusion globale

Cette première étape a permis de comprendre la structure du corpus et d'extraire les thèmes principaux des conversations patient-thérapeute. Le pré-traitement rigoureux (filtrage des langues, lemmatisation, stopwords) a été déterminant pour la qualité des résultats de topic modeling.

Les différentes méthodes explorées (TF-IDF + K-Means, Word2Vec, Sentence Transformer, LDA) ont chacune leurs avantages et limites, et ont permis d'obtenir des perspectives complémentaires sur les thèmes abordés dans les conversations. Cependant, des problèmes techniques (hétérogénéité du nettoyage, gestion des doublons) ainsi que des questions ouvertes (standardisation du nettoyage, identification des thérapeutes) restent à résoudre pour améliorer la qualité et la comparabilité des résultats.

Maintenant que nous avons une bonne compréhension des thèmes présents dans les conversations, nous allons pouvoir passer à l'étape suivante qui consiste à analyser les sentiments exprimés et à implémenter un système de question-réponse sur la santé mentale.

Troisième partie

Étape 2 — Analyse et Accès aux conversations

Chapitre 8

Vue d'ensemble

8.1 Étape 2 — Analyse et Accès aux conversations

Cette seconde étape constitue le cœur applicatif du projet. Elle se divise en **deux modules distincts mais complémentaires**, développés en parallèle par les deux sous-groupes de l'équipe. Le premier module vise à doter le système d'une capacité de reconnaissance automatique des émotions, tandis que le second exploite ces annotations pour construire un moteur de recherche de réponses adaptées au contexte psychologique de l'utilisateur.

8.1.1 Module 1 — Classification automatique des sentiments

Objectif : Attribuer automatiquement une étiquette de sentiment à chaque échange patient-thérapeute, selon une taxonomie clinique de sept catégories (Anxiety, Bipolar, Depression, Normal, Personality disorder, Stress, Suicidal), en se basant exclusivement sur le contenu textuel.

Corpus d'entraînement. Le modèle est entraîné sur le dataset `Combined_Data.csv` (*Sentiment Analysis for Mental Health*, Kaggle), un corpus annoté indépendant du jeu de données principal.

Approches implémentées. Deux méthodes d'apprentissage sont explorées et comparées :

- **Méthodes supervisées** — Naive Bayes, Régression Logistique et LinearSVC exploitent les annotations manuelles pour apprendre une fonction de décision, en s'appuyant sur les représentations vectorielles issues de l'Étape 1 (TF-IDF, Word2Vec).
- **Méthode non supervisée** — L'algorithme K-Means partitionne l'espace des caractéristiques sans recours aux étiquettes ; les clusters obtenus sont ensuite mis en correspondance avec les catégories cliniques.

Pipeline d'évaluation. Chaque méthode suit un protocole standardisé : découpage stratifié en ensembles d'entraînement (60%), de validation (20%) et de test (20%), puis évaluation quantitative via accuracy, précision, rappel, F1-score et matrice de confusion. Les modèles les plus performants sont déployés sur le corpus principal (`train.csv`) pour annotation automatique.

Transfert inter-modules. Les annotations produites sont transmises au Module 2 sous forme tabulaire (identifiant de conversation, étiquette prédite, score de confiance), permettant un filtrage par cohérence émotionnelle des réponses retournées.

Remarque

Les résultats détaillés des approches classiques sont présentés au Chapitre 9, tandis que les approches avancées basées sur BERT font l'objet du Chapitre 10.

8.1.2 Module 2 — Moteur de question-réponse orienté santé mentale

Objectif : À partir d'une question formulée en langage naturel, retourner les k réponses les plus pertinentes extraites du corpus de conversations thérapeutiques, classées par score de similarité décroissant.

Architecture de l'index. Le système construit un **index inversé** à partir du corpus nettoyé : chaque terme du vocabulaire est associé à l'ensemble des paires question-réponse qui le contiennent, permettant une récupération rapide des conversations candidates.

Stratégies de recherche. Deux approches sont implémentées et comparées :

- **Similarité question-question (Q-Q)** — Identifier les k questions du corpus les plus proches de la requête, puis retourner les réponses associées.
- **Similarité question-réponse (Q-R)** — Rechercher directement les k réponses du corpus maximisant la similarité sémantique avec la question posée.

La similarité est calculée par la **mesure du cosinus** appliquée à trois types de représentations vectorielles couvrant un spectre du lexical au sémantique : TF-IDF, Word2Vec et BERT (all-MiniLM-L6-v2).

Intégration du filtrage par sentiment. Le module exploite les annotations du Module 1 pour restreindre l'espace de recherche aux conversations partageant l'étiquette de sentiment de la question d'entrée. Ce mécanisme de **filtrage par cohérence émotionnelle** assure que les réponses retournées sont non seulement pertinentes lexicalement, mais également adaptées au contexte psychologique de la requête.

Protocole d'évaluation. Les performances sont mesurées sur un jeu de test de 10 paires question-réponse de référence, via la moyenne des similarités cosinus entre réponses prédites et réponses attendues. Les deux versions (avec et sans filtrage par sentiment) sont comparées pour quantifier l'apport des annotations émotionnelles à la qualité des résultats.

Remarque

Les détails du calcul de similarité, l'implémentation, les résultats complets et l'analyse comparative des stratégies de recherche sont présentés au Chapitre 11.

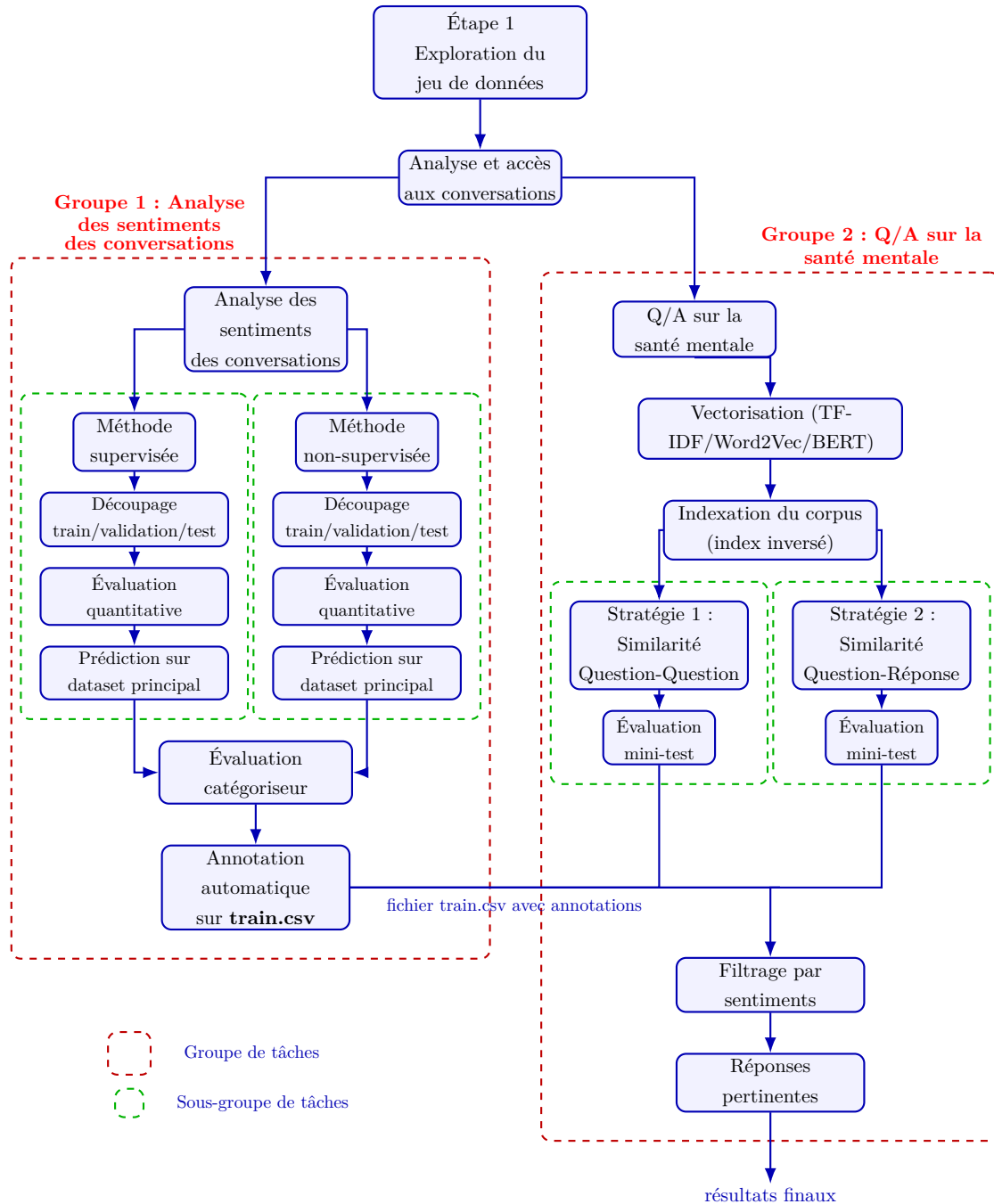


FIGURE 8.1 – Pipeline détaillé de la seconde étape : Analyse et accès aux conversations

Chapitre 9

Module 1 : Analyse des sentiments — Approches classiques

9.1 Contexte et organisation

L'étape 2 a divisé le groupe 1 en deux sous-groupes travaillant en parallèle sur le même jeu de données d'entraînement (`combined_data.csv`, *Sentiment Analysis for Mental Health* sur Kaggle), avec pour objectif commun d'annoter automatiquement le dataset principal `train.csv` en vue de sa transmission au Groupe 2.

Sous-groupe	Méthode	Membres
Supervisé	Naive Bayes (BoW + TF-IDF) + Logistic Regression (TF-IDF) + LinearSVC (BoW + TF-IDF)	Diogo, Ulysse, Logan
Non-supervisé	K-Means (Word2Vec + TF-IDF)	Emin, Victor

Le pipeline commun aux deux sous-groupes comprend : nettoyage du texte, tokenisation (NLTK + Porter Stemmer), et vectorisation (BoW, TF-IDF, Word2Vec). Le split des données est identique pour les deux : **60 % train / 20 % validation / 20 % test**, avec stratification sur les classes.

9.1.1 Différence entre méthode supervisée et non-supervisée

Une méthode supervisée apprend à partir d'**exemples déjà étiquetés**. Pour chaque entrée, on connaît la bonne réponse pendant l'entraînement ; le modèle apprend donc à associer des entrées à des sorties connues.

Exemple : classer des e-mails en spam / non-spam, prédire un prix, détecter une maladie à partir de symptômes.

Une méthode non-supervisée, quant à elle, apprend à partir de **données non étiquetées**. Il n'existe pas de bonne réponse fournie au préalable ; le but est de découvrir des structures cachées, des groupes ou des patterns dans les données.

Exemple : regrouper des clients en segments, détecter des anomalies, réduire la dimension des données.

En résumé :

- Supervisée → on connaît la réponse attendue.
- Non-supervisée → on ne connaît pas la réponse, on cherche des structures.

9.1.2 Métriques d'évaluation utilisées

Afin de comparer les modèles et sélectionner le meilleur pour annoter `train.csv`, nous distinguons trois catégories de métriques selon leur domaine d'application.

Métriques communes aux deux méthodes

Ces deux métriques s'appliquent aussi bien à la méthode supervisée qu'à la méthode non-supervisée (après association des clusters aux labels pour cette dernière), ce qui en fait les seuls indicateurs permettant une **comparaison directe** entre les deux approches.

Accuracy Pourcentage de sentiments correctement prédits :

$$\text{Accuracy} = \frac{\text{Nb. de bonnes prédictions}}{\text{Nb. total d'exemples}}$$

Matrice de confusion Permet de visualiser les prédictions par rapport aux vraies classes. Les lignes représentent les vraies classes, les colonnes les classes prédites. Elle révèle les erreurs systématiques du modèle : quelles classes sont confondues entre elles, et dans quelle direction.

Métriques propres à la méthode supervisée

Ces métriques nécessitent de connaître les vrais labels pour chaque exemple, ce qui est possible en apprentissage supervisé mais pas directement applicable au clustering non-supervisé.

MSE (Mean Squared Error) Erreur moyenne au carré entre la vraie valeur et la valeur prédite :

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

R² (Coefficient de détermination) Mesure la proportion de variance des données expliquée par le modèle :

$$R^2 = 1 - \frac{\text{Erreur modèle}}{\text{Erreur moyenne}}$$

Attention

MSE et R^2 sont des métriques de régression. Elles supposent qu'il existe un ordre et une distance numérique significative entre les valeurs prédites et les valeurs réelles. Or nos labels sont nominaux (*Depression*, *Anxiety*, *Normal*...) encodés arbitrairement en entiers : la "distance" entre deux classes n'a aucun sens clinique ni mathématique. Ces deux métriques ne sont donc **pas des indicateurs fiables** pour ce problème et ne doivent pas être utilisées comme critères de décision. L'accuracy et le F1-score sont à privilégier.

Precision, Recall et F1-score Ces trois métriques issues du classification report permettent d'évaluer les performances **classe par classe**, ce qui est essentiel ici en raison du fort déséquilibre entre les 7 catégories de sentiments.

— **Precision** — parmi les prédictions positives, combien sont vraiment positives ?

$$\text{Precision} = \frac{TP}{TP + FP}$$

— **Recall** — parmi les vrais positifs, combien ont été retrouvés ?

$$\text{Recall} = \frac{TP}{TP + FN}$$

— **F1-score** — compromis entre precision et recall, particulièrement utile sur des classes déséquilibrées :

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Avec : TP = vrai positif, TN = vrai négatif, FP = faux positif, FN = faux négatif.

Remarque

Pour comparer les méthodes supervisées entre elles (NB vs LR), nous privilégions le **F1-score macro** (moyenne non pondérée sur les classes) plutôt que le F1 *weighted*, afin de ne pas masquer les mauvaises performances sur les classes minoritaires (*Stress*, *Personality disorder*).

Métriques propres à la méthode non-supervisée

Ces métriques sont spécifiques au clustering : elles évaluent la qualité des groupes formés **sans faire référence aux vrais labels** pendant l'entraînement, ce qui les rend non comparables directement avec les métriques supervisées.

Purity globale Mesure la cohérence interne des clusters : pour chaque cluster, on retient uniquement le label majoritaire parmi ses membres.

$$\text{Purity} = \frac{1}{N} \sum_k \max_j |C_k \cap L_j|$$

Avec N le nombre total d'exemples, C_k le cluster k et L_j la classe réelle j .

Remarque

Plus la Purity est proche de 1, meilleur est le regroupement. Attention : si chaque point forme son propre cluster, la purity vaut 1 — cette mesure peut donc être trompeuse si utilisée seule.

ARI (Adjusted Rand Index) Mesure la similarité entre les vrais labels et les clusters trouvés, en corrigeant le score pour tenir compte du hasard. Sa valeur est comprise entre -1 (pire qu'aléatoire) et 1 (clustering parfait). Contrairement à la Purity, l'ARI **pénalise un trop grand nombre de clusters** et les mauvais regroupements, ce qui en fait un indicateur plus robuste.

Tableau récapitulatif

Catégorie	Mesure	Supervisée	Non-supervisée
Communes	Accuracy	Oui	Oui (après mapping)
	Matrice confusion	Oui	Oui
Supervisée uniquement	MSE	Oui	Non
	R^2	Oui	Non
	Precision	Oui	Non
	Recall	Oui	Non
	F1-score	Oui	Non
Non-supervisée uniquement	Purity	Non	Oui
	ARI	Non	Oui

9.2 Résultats par membre

9.2.1 Méthode supervisée — Naive Bayes et Logistic Regression

Qu'est-ce que Naive Bayes ?

Naive Bayes répond à la question : « **Étant donné les mots de ce texte, quelle est la catégorie la plus probable ?** »

C'est un classificateur probabiliste : plutôt que de tracer une frontière entre les classes comme le ferait un SVM, il **calcule une probabilité** pour chaque classe et retourne celle qui est la plus élevée, en s'appuyant sur le théorème de Bayes :

$$P(\text{classe} \mid \text{mots}) \propto P(\text{classe}) \times \prod_i P(\text{mot}_i \mid \text{classe})$$

Le modèle fait une hypothèse simplificatrice : chaque mot est supposé **indépendant** des autres. Cette simplification rend l'algorithme très rapide à entraîner et étonnamment efficace sur les textes malgré son aspect « naïf ».

Qu'est-ce que la Logistic Regression ?

La Régression Logistique répond à la question : « **Étant donné ce vecteur TF-IDF, à quelle classe ce texte appartient-il le plus probablement ?** »

Contrairement à Naive Bayes qui calcule des probabilités indépendamment mot par mot, la Régression Logistique **apprend un poids pour chaque feature** (ici chaque n -gram TF-IDF) et combine ces poids pour produire un score par classe.

	Naive Bayes	Logistic Regression
Hypothèse	Mots indépendants	Aucune hypothèse sur les features
Frontière	Probabiliste naïve	Frontière de décision optimisée
Classes déséquilibrées	Biais vers la classe majoritaire	Géré via <code>class_weight='balanced'</code>
Corrélations entre mots	Ignorées	Prises en compte implicitement

Le paramètre clé de la LR est **C** (inverse de la régularisation) : un **C** petit entraîne une régularisation forte (modèle simple, évite l'overfitting) ; un **C** grand entraîne une régularisation faible (modèle plus complexe, peut overfit).

Qu'est-ce que LinearSVC ?

Le LinearSVC répond à la question : « **Quelle est la frontière optimale qui sépare le mieux les classes ?** »

Contrairement à la Régression Logistique qui estime des probabilités, le LinearSVC cherche un **hyperplan** (une droite en 2D, un plan en 3D, généralisé en N dimensions) qui sépare les classes en **maximisant la marge**, c'est-à-dire la distance entre la frontière et les points les plus proches de chaque classe. Ces points les plus proches sont les **vecteurs de support** : ils sont les seuls à définir la frontière, tous les autres exemples n'ont aucune influence.

La frontière est linéaire, ce qui est parfaitement adapté aux vecteurs TF-IDF déjà en haute dimension. **En haute dimension, les classes sont souvent linéairement séparables même si elles ne le seraient pas en 2D.**

	Naive Bayes	Logistic Regression	LinearSVC
Approche	Probabiliste	Probabiliste	Géométrique
Apprend	$P(\text{classe} \text{mots})$	Poids par feature	Frontière à marge max
Sortie	Probabilité	Probabilité	Score de décision
Déséquilibre	Sensible	Géré via <code>class_weight</code>	Géré via <code>class_weight</code>
Vitesse	Très rapide	Rapide	Très rapide (haute dim.)
Atout principal	Simplicité	Calibration	Marge max →meilleure généralisation

Remarque

LinearSVC est souvent le modèle le plus performant sur des tâches de classification de texte TF-IDF, car il est nativement conçu pour les espaces de haute dimension et les données creuses (*sparse*), ce qui correspond exactement aux vecteurs TF-IDF.

Diogo et Ulysse — Naive Bayes BoW et TF-IDF

Diogo et Ulysse ont produit les résultats de référence pour la méthode supervisée.

Tuning de l'hyperparamètre α (lissage de Laplace) :

- Valeurs testées : [0.001, 0.01, 0.05, 0.1, 0.5, 1.0, 2.0, 5.0]
- Meilleur α BoW : **0.1** (accuracy validation : 0.6606)
- Meilleur α TF-IDF : **0.05** (accuracy validation : 0.6761)

Résultats sur l'ensemble test :

Modèle	Accuracy test
Naive Bayes + BoW	0.6666
Naive Bayes + TF-IDF	0.6875

Classification report (NB + BoW) :

Classe	Precision	Recall	F1-score
Anxiety	0.68	0.73	0.71
Bipolar	0.60	0.76	0.68
Depression	0.57	0.61	0.59
Normal	0.92	0.70	0.80
Personality disorder	0.50	0.56	0.53
Stress	0.52	0.43	0.47
Suicidal	0.60	0.72	0.65

Classification report (NB + TF-IDF) :

Classe	Precision	Recall	F1-score
Anxiety	0.76	0.71	0.73
Bipolar	0.82	0.58	0.68
Depression	0.54	0.77	0.63
Normal	0.90	0.77	0.83
Personality disorder	0.97	0.29	0.45
Stress	0.83	0.27	0.41
Suicidal	0.65	0.60	0.62

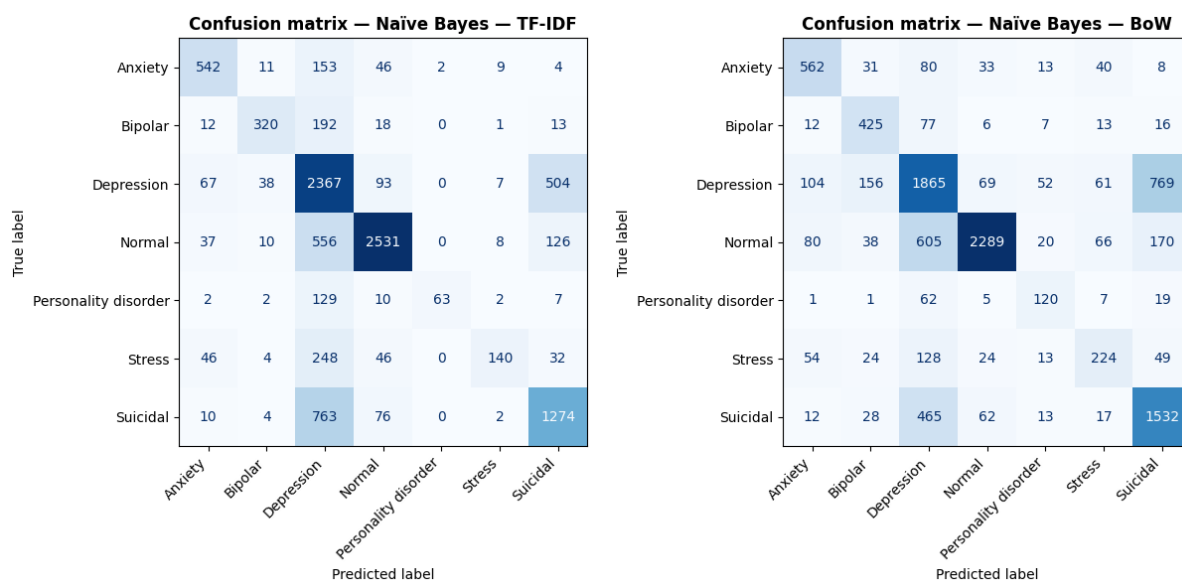


FIGURE 9.1 – Matrice de confusion — NB + TF-IDF (gauche) et NB + BoW (droite) (Diogo et Ulysse)

Remarque

TF-IDF est meilleur sur les deux classes les plus représentées (*Normal* et *Depression*), ce qui tire son accuracy globale vers le haut. Mais sur les 5 autres classes, **BoW gagne à chaque fois**, parfois très largement : *Stress* (43 % vs 27 % de recall) ou *Bipolar* (76 % vs 58 % de recall).

Pourquoi ? TF-IDF pénalise les mots fréquents. C'est utile pour séparer *Normal* de *Depression*, mais pour les classes rares (*Stress*, *Personality disorder*), les mots caractéristiques sont précisément des mots fréquents dans le corpus global (*overwhelmed*, *pressure*, *relationship*...). TF-IDF les « dépouille » de leur signal. Le cas de *Personality disorder* est frappant : **precision 0.97 mais recall 0.29** — le modèle est très sûr quand il prédit cette classe, mais il rate 71 % des vrais cas.

Logan — Naive Bayes + Logistic Regression

Logan a produit le notebook de base structurant le travail commun. Sa contribution individuelle principale est l'implémentation d'une **Régression Logistique** (LR + TF-IDF optimisé), identifiée comme « version extra-performante ».

Paramétrage TF-IDF amélioré :

```
tfidf_vectorizer = TfidfVectorizer(
    ngram_range=(1, 3),      # unigrammes, bigrammes et trigrammes
    max_features=50000,      # limite aux 50k features les plus
                             utiles
    sublinear_tf=True,       # log(tf) au lieu de tf
    min_df=2,                # ignore les mots < 2 documents
    max_df=0.95              # ignore les mots > 95% des documents
)
```

Tuning de l'hyperparamètre alpha (Naive Bayes) :

- Valeurs testées : [0.001, 0.005, 0.01, 0.03, 0.05, 0.07, 0.1]
- Meilleur alpha BoW : **0.1** (accuracy : 0.6606)
- Meilleur alpha TF-IDF : **0.07** (accuracy : 0.6816)

Tuning de l'hyperparamètre C (Logistic Regression) :

- Valeurs testées : [0.01, 0.05, 0.1, 0.3, 0.5, 1.0, 3.0, 5.0, 10.0]
- Meilleur C : **3.0** (accuracy validation : 0.7457)

Résultats comparés sur l'ensemble test :

Modèle	Accuracy test
Naive Bayes + BoW	0.6666
Naive Bayes + TF-IDF	0.6927
Logistic Regression + TF-IDF	0.7532

Comparaison F1-score NB + TF-IDF vs LR + TF-IDF :

Classe	F1 NB TF-IDF	F1 LR TF-IDF	Évolution
Anxiety	0.73	0.80	+0.07
Bipolar	0.67	0.79	+0.16
Depression	0.63	0.69	+0.06
Normal	0.85	0.90	+0.15
Personality disorder	0.40	0.66	+0.26
Stress	0.37	0.57	+0.20
Suicidal	0.63	0.64	+0.01

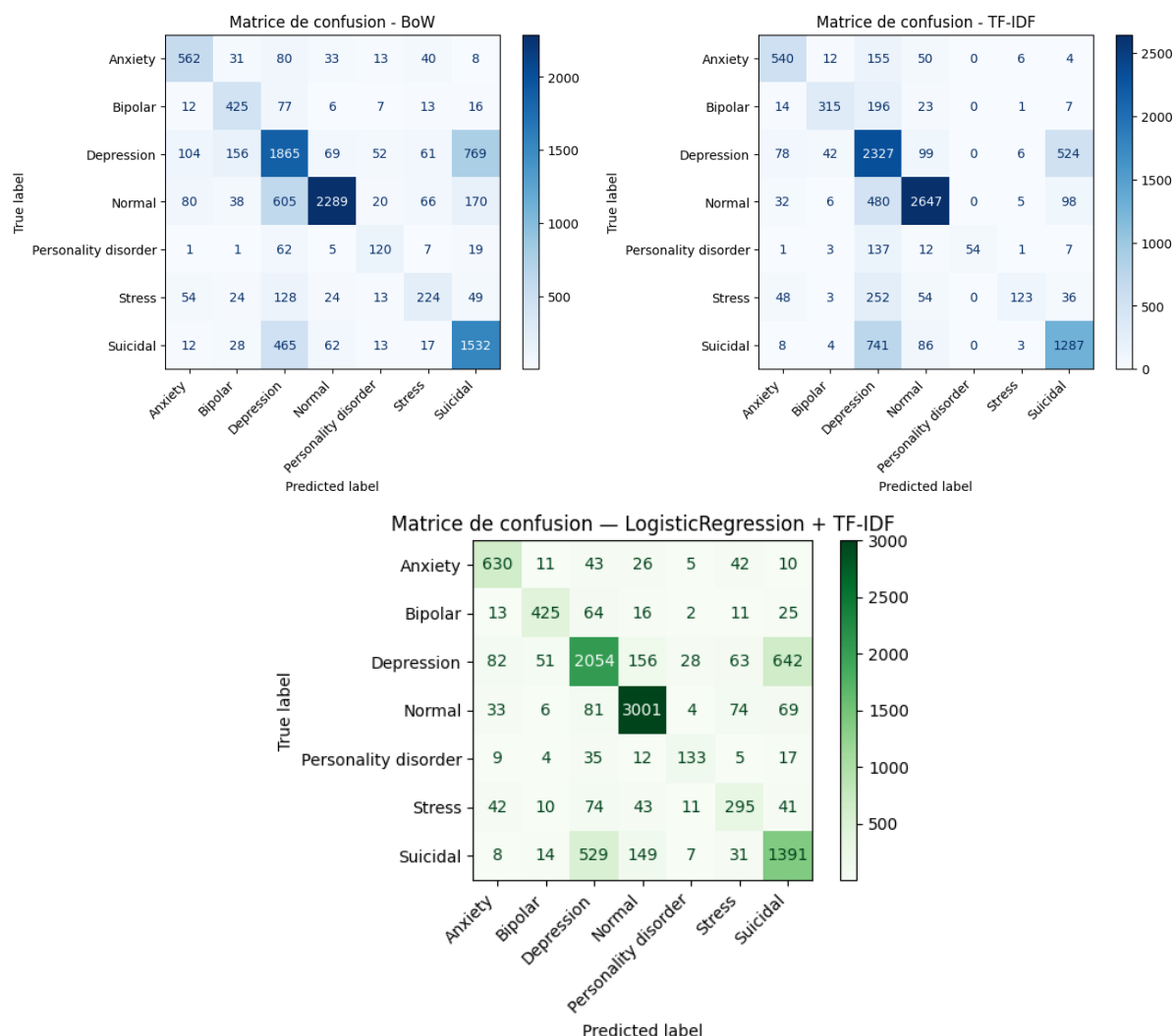


FIGURE 9.2 – Matrice de confusion — NB + BoW (Haut-Gauche) et NB + TF-IDF (Haut-Droite) et LR + TF-IDF (Bas) (Logan)

Remarque

La Logistic Regression améliore le F1-score sur **toutes les classes**, en particulier *Personality disorder* (+0.26) et *Stress* (+0.20) grâce au paramètre `class_weight='balanced'` qui compense le déséquilibre du dataset.

Problème critique

Classe *Suicidal* : sur 2129 vrais cas *Suicidal*, 1391 sont correctement classés **mais** 529 sont prédits *Depression*. Confondre *Suicidal* avec *Depression* peut avoir des conséquences réelles dans un contexte médical. Ce problème est commun à toutes les approches testées.

Diogo — LinearSVC + BoW et TF-IDF

Après l'implémentation de Naive Bayes et Logistic Regression, Diogo a exploré le **LinearSVC** (*Linear Support Vector Classifier*), une méthode géométriquement très différente des approches probabilistes précédentes.

Tuning de l'hyperparamètre C :

- Valeurs testées : [0.01, 0.1, 1.0]
- Meilleur C BoW : **0.1** (accuracy : 0.7309)
- Meilleur C TF-IDF : **1.0** (accuracy : 0.7386)

Avec seulement 3 valeurs testées, le temps de traitement atteint déjà **8 minutes**. Une stratégie d'optimisation *coarse-to-fine* est envisageable : une *coarse search* sur une large plage (ex. : 0.001, 0.01, 0.1, 1, 10, 100) pour repérer la bonne zone, puis une *fine search* plus précise autour des meilleures valeurs trouvées.

Classification report (LinearSVC + BoW) :

Classe	Precision	Recall	F1-score
Anxiety	0.77	0.73	0.75
Bipolar	0.79	0.73	0.76
Depression	0.71	0.65	0.68
Normal	0.84	0.95	0.89
Personality disorder	0.68	0.59	0.63
Stress	0.52	0.46	0.49
Suicidal	0.63	0.63	0.63

Classification report (LinearSVC + TF-IDF) :

Classe	Precision	Recall	F1-score
Anxiety	0.81	0.79	0.80
Bipolar	0.86	0.71	0.78
Depression	0.68	0.70	0.69
Normal	0.85	0.94	0.89
Personality disorder	0.86	0.56	0.68
Stress	0.68	0.46	0.55
Suicidal	0.63	0.61	0.62

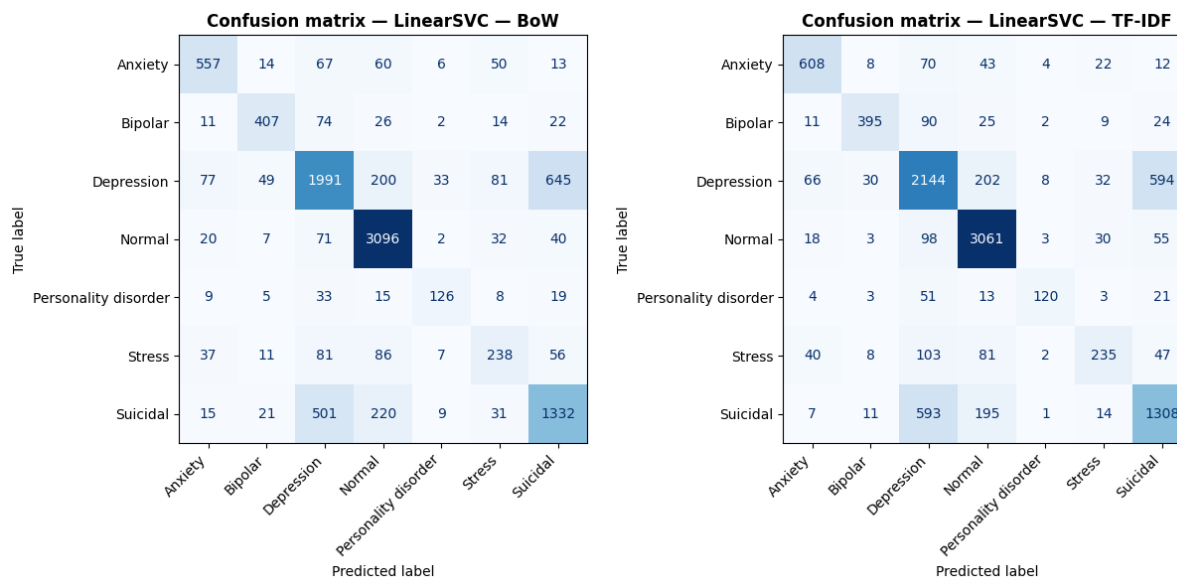


FIGURE 9.3 – Matrice de confusion — LinearSVC + BoW (gauche) et LinearSVC + TF-IDF (droite) (Diogo)

Analyse détaillée des confusions :

Modèle	Vrais <i>Suicidal</i>	Vrais <i>Depression</i>
	→ prédits <i>Depression</i>	→ prédits <i>Suicidal</i>
LinearSVC TF-IDF	593	594
LinearSVC BoW	501	645

TF-IDF est légèrement meilleur sur les deux sens de la confusion *Depression/Suicidal*, ce qui est cohérent avec son meilleur F1 global sur ces deux classes.

Modèle	Vrais <i>Personality disorder</i> classés	Prédits <i>Depression</i>
LinearSVC TF-IDF	120/215 → 56 %	51
LinearSVC BoW	126/215 → 59 %	33

Remarque

Résultat contre-intuitif : BoW retrouve mieux les vrais *Personality disorder* malgré une accuracy globale inférieure. TF-IDF les noie davantage dans *Depression*.

Modèle	Vrais <i>Stress</i> classés	Prédits <i>Depression</i>	Prédits <i>Normal</i>
LinearSVC TF-IDF	235/516 → 46 %	103	81
LinearSVC BoW	238/516 → 46 %	81	86

Les deux modèles sont quasi identiques sur *Stress*, mais BoW disperse moins vers *Normal* et TF-IDF disperse moins vers *Depression*.

Comparaison F1-score entre toutes les méthodes supervisées (vectorisation TF-IDF) :

Classe	NB TF-IDF	LR TF-IDF	LinearSVC TF-IDF	Meilleur
Anxiety	0.73	0.80	0.80	LR et LinearSVC
Bipolar	0.68	0.79	0.78	LR
Depression	0.63	0.69	0.69	LR et LinearSVC
Normal	0.83	0.90	0.89	LR
Personality disorder	0.45	0.66	0.68	LinearSVC
Stress	0.41	0.57	0.55	LR
Suicidal	0.62	0.64	0.62	LR

LinearSVC et LR sont quasi-équivalents sur la majorité des classes : l'écart est souvent de 0.01 à 0.02 de F1, dans la marge de variabilité normale. Le seul point où LinearSVC prend l'avantage est *Personality disorder* (0.68 vs 0.66), la classe la plus difficile du dataset.

NB reste nettement distancé sur toutes les classes minoritaires. Cet écart est structurel : NB suppose l'indépendance des mots, ce qui lui fait rater les signaux contextuels que LR et LinearSVC captent tous les deux.

La confusion *Depression/Suicidal* persiste quel que soit le modèle. Ce n'est donc pas un problème algorithmique mais une limite du dataset, ces deux classes partageant un vocabulaire très proche.

9.2.2 Méthode non-supervisée — K-Means (Emin et Victor)

Qu'est-ce que K-Means ?

L'algorithme K-Means regroupe les textes en calculant la **proximité mathématique** entre les vecteurs. Il place des centres (centroïdes) de manière aléatoire, puis les déplace itérativement jusqu'à ce que chaque texte soit rattaché au groupe le plus proche, minimisant ainsi l'inertie totale.

Le sous-groupe non-supervisé a comparé deux approches de vectorisation : **TF-IDF** + **SVD** et **Word2Vec**.

Victor — Implémentation Word2Vec

Victor a privilégié l'utilisation de **Word2Vec** pour capturer le sens profond des mots. Contrairement au TF-IDF qui ne fait que compter, Word2Vec comprend que des termes comme *triste* et *déprimé* sont sémantiquement proches.

Sélection de k : le nombre de groupes (entre 2 et 15) a été choisi automatiquement via le **score de silhouette cosinus** sur la validation, avec un plancher à `max(best_k, y_train.nunique())`.

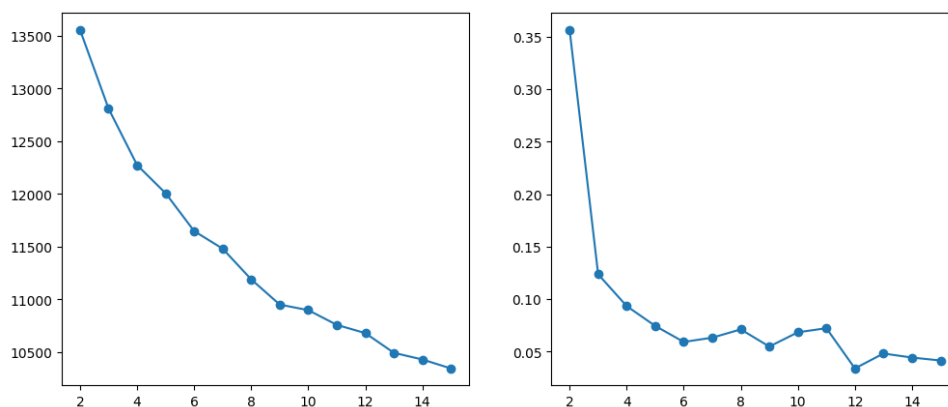


FIGURE 9.4 – Courbes WCSS et silhouette pour la sélection de k (Victor)

Mapping cluster $\rightarrow$ label (vote majoritaire) : pour nommer chaque cluster, on identifie la classe réelle dominante parmi ses membres. En cas de conflit, le cluster de plus haute pureté conserve l'étiquette (stratégie gloutonne).

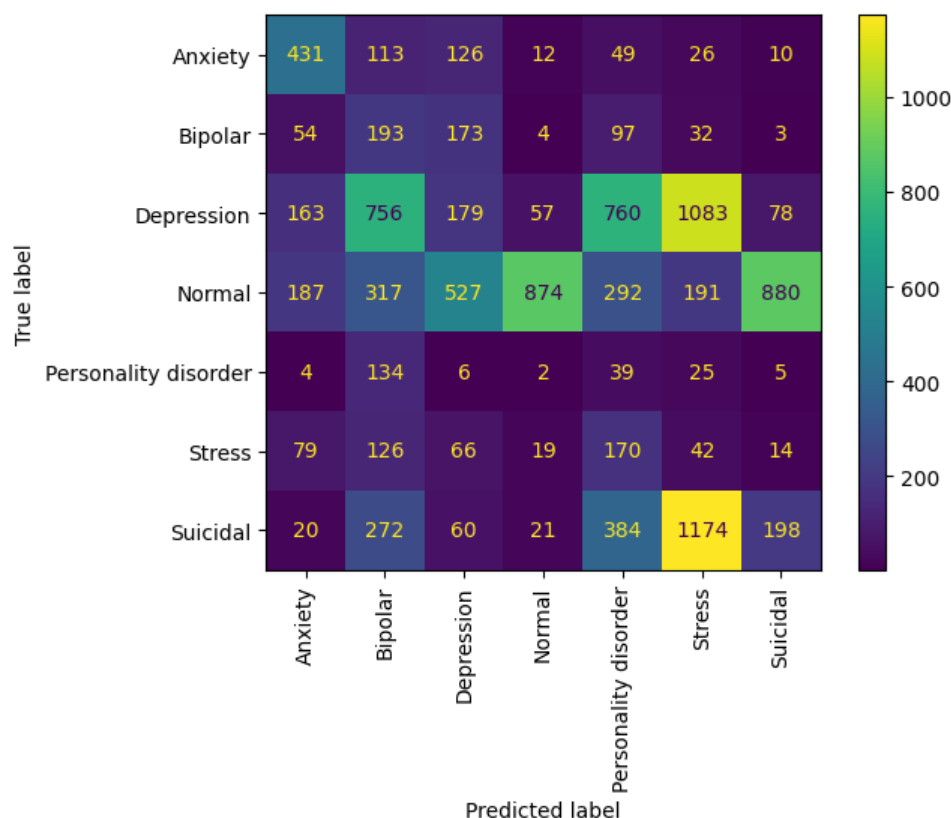


FIGURE 9.5 – Résultat du mapping et pureté des clusters (Victor)

Remarque

Victor a corrigé une erreur du template qui tentait de mélanger TF-IDF et Word2Vec dans la cellule de déploiement final, ce qui rendait le pipeline incohérent.

Emin — Comparaison TF-IDF et robustesse du pipeline

Emin a complété l'étude en implémentant le pipeline complet sous **TF-IDF + SVD** pour comparer les deux représentations.

Pourquoi utiliser SVD avec TF-IDF ? Le possible problème que l'on peut avoir ici est que les vecteurs TF-IDF sont très creux (beaucoup de zéros) et en très haute dimension (50 000 features). Cela peut rendre la tâche de clustering plus difficile pour K-Means, qui fonctionne mieux dans des espaces denses et de dimension modérée. En appliquant une réduction de dimensionnalité via SVD (*TruncatedSVD* sous python), on peut compacter l'information essentielle dans un espace plus petit (ex. : 100 dimensions), ce qui facilite le travail de K-Means.

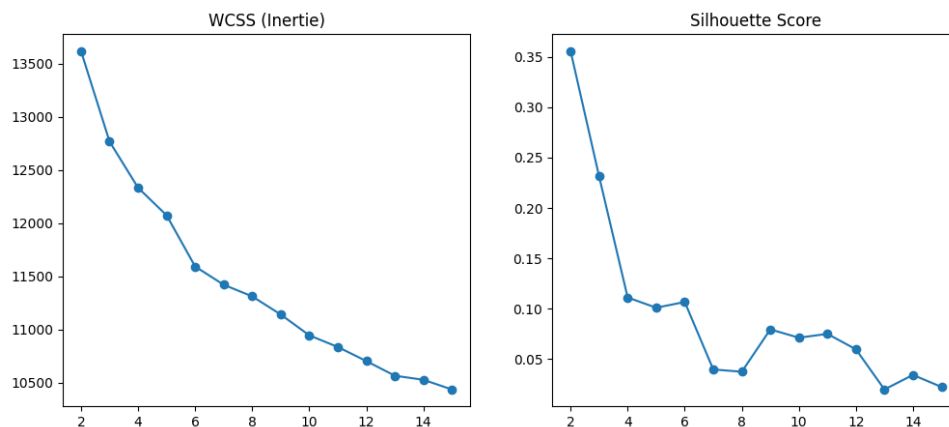
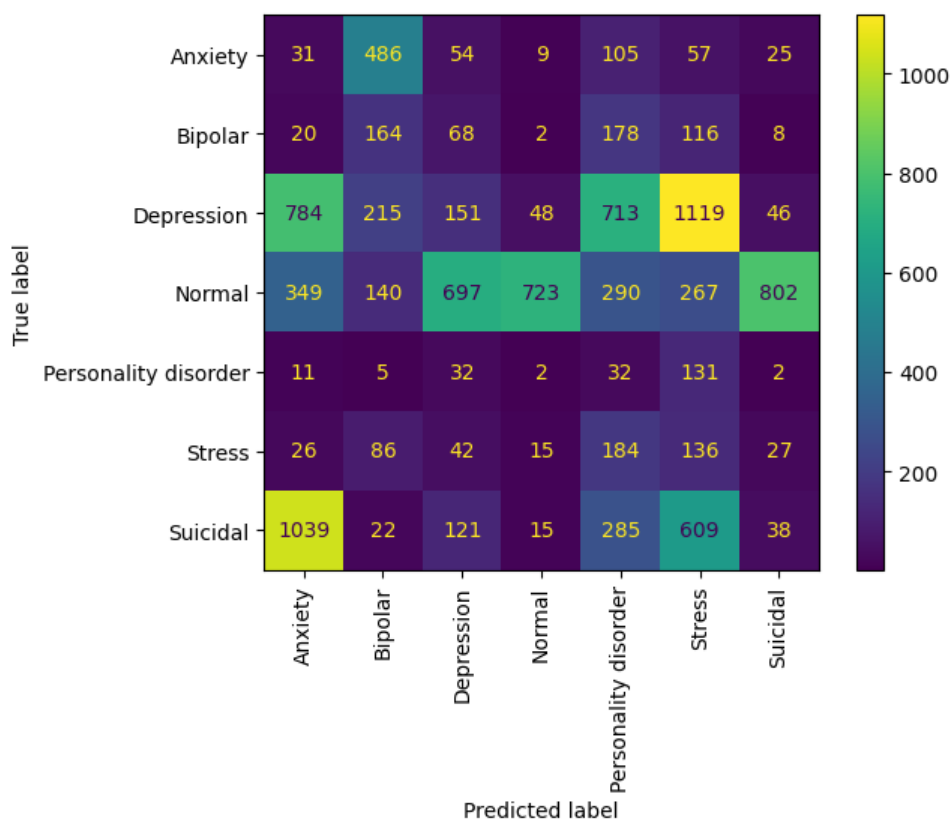
FIGURE 9.6 – Courbes WCSS et silhouette pour la sélection de k (Emin)

FIGURE 9.7 – Résultat du mapping et pureté des clusters (Emin)

Emin a également résolu des conflits de versions entre `gensim` et `scipy` dans l'environnement de travail, et standardisé la cellule de déploiement B.6 pour utiliser le modèle TF-IDF.

Synthèse — Comparaison TF-IDF vs Word2Vec pour K-Means

Critère	K-Means + TF-IDF	K-Means + Word2Vec
Meilleur k retenu	7	7
Accuracy test	0.4636	0.20
Purity globale	0.54	0.51
ARI	0.25	0.13
Besoin de SVD ?	Oui	Non
Dimension de l'espace	100 (après SVD)	100 (natif)
Pureté max cluster	0.85 (<i>Depression</i>)	0.90 (<i>Normal</i>)
Pureté min cluster	0.30 (<i>Personality disorder</i>)	0.30 (<i>Bipolar</i>)

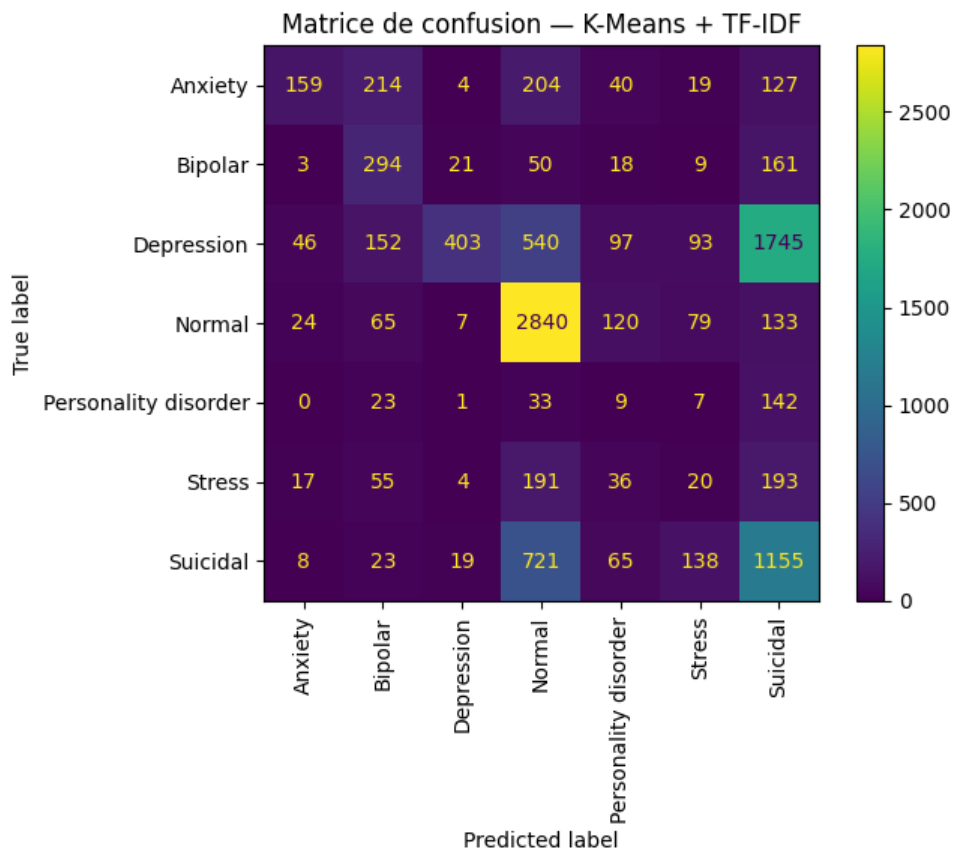


FIGURE 9.8 – Matrice de confusion — K-Means final (TF-IDF vs Word2Vec)

Remarque

Contra-intuitivement, TF-IDF (46 %) surpasse Word2Vec (20 %). Les catégories cliniques du dataset (*Depression*, *Anxiety*, etc.) se reconnaissent avant tout par un **vocabulaire symptomatique spécifique**, ce que TF-IDF met précisément en avant. Word2Vec, qui regroupe les mots par contexte sémantique large, tend à fusionner *Depression* et *Suicidal* dans les mêmes clusters — le mapping glouton force alors ces gros clusters sur des labels minoritaires, effondrant l’accuracy globale.

Modèle retenu pour le non-supervisé : K-Means + TF-IDF.

Fichier produit : `train_with_sentiments_kmeans.csv`.

9.3 Comparaison globale : supervisé vs. non-supervisé

Critère	NB TF-IDF (supervisé)	K-Means TF-IDF (non-supervisé)
Accuracy test	0.6875	0.4636
Macro avg F1	0.62	—
Weighted avg F1	0.69	—
Voit les labels à l'entraînement ?	Oui	Non
Hyperparamètre clé	alpha = 0.05	k = 7
Fichier annoté	train_with_ sentiments.csv	train_with_ sentiments_kmeans.csv

L'écart de **+22 points d'accuracy** en faveur du supervisé est cohérent et attendu : le supervisé a accès aux vrais labels pendant l'entraînement, le non-supervisé pas. Les deux méthodes utilisent la même représentation TF-IDF, donc l'écart reflète purement la différence supervisé/non-supervisé.

Remarque

Les mêmes classes sont difficiles pour les deux approches (*Personality disorder*, *Stress*, confusion *Depression/Suicidal*). Cela pointe vers une **limite des données elles-mêmes** (classes sous-représentées, vocabulaire qui se chevauche) plutôt qu'un problème algorithmique.

La prochaine partie introduit une nouvelle famille de modèles basés sur les **Transformers**, qui pourraient potentiellement mieux capturer les subtilités du langage clinique et améliorer les performances, notamment sur les classes difficiles.

Chapitre 10

Module 1 EXTRA : Analyse des sentiments — Approches BERT

10.1 Qu'est-ce que BERT ?

10.1.1 Définition générale

BERT (*Bidirectional Encoder Representations from Transformers*) est un modèle de langage développé par Google en 2018. Il repose sur l'architecture **Transformer**, mais n'en utilise que la partie **encodeur**.

Concrètement, un encodeur transforme une séquence de tokens en une séquence de vecteurs denses appelés **représentations contextuelles** : chaque token se voit attribuer un vecteur dont le contenu dépend de l'ensemble de la phrase (et non du seul mot considéré). Ce mécanisme repose sur la **self-attention multi-têtes**, qui permet à chaque token de "regarder" tous les autres tokens simultanément et de pondérer leur importance.

BERT empile ainsi 12 couches (version *base*) ou 24 couches (version *large*) de ce bloc encodeur, ce qui lui permet de construire des représentations de plus en plus abstraites et nuancées au fil des couches. Le décodeur (utilisé pour générer du texte, comme dans GPT) est délibérément absent : BERT produit une **compréhension** du texte, pas une génération.

De plus, sa particularité fondamentale est d'être **bidirectionnel** : contrairement aux modèles précédents qui lisaient le texte uniquement de gauche à droite (ou de droite à gauche), BERT lit le contexte dans les deux directions simultanément.

Exemple d'ambiguïté résolue par la bidirectionnalité

« Le chat mange la souris »

BERT comprend le mot *mange* en regardant à la fois *chat* (contexte gauche) et *souris* (contexte droit) en une seule passe — ce qu'un modèle unidirectionnel ne peut pas faire.

10.1.2 Étapes d'apprentissage

L'apprentissage de BERT se déroule en deux phases distinctes.

1. Pré-entraînement (non supervisé)

Avant toute tâche spécifique, BERT est pré-entraîné sur d'énormes corpus (Wikipedia anglophone, BooksCorpus, etc.) via deux tâches auto-supervisées :

- **Masked Language Model (MLM)** : environ 15 % des tokens sont masqués aléatoirement et le modèle doit les prédire à partir du contexte. Exemple : « Le [MASK] mange la souris » → prédit chat.
- **Next Sentence Prediction (NSP)** : le modèle apprend à déterminer si deux phrases se suivent logiquement dans un texte ou sont juxtaposées aléatoirement.

Cette phase est entièrement **non supervisée** : aucune annotation humaine n'est requise. Elle permet à BERT d'acquérir une compréhension riche et générale du langage.

2. Fine-tuning supervisé

C'est ici qu'intervient l'apprentissage **supervisé**. On reprend le BERT pré-entraîné et on l'adapte à une tâche précise avec des données étiquetées, en ajoutant une petite couche de sortie :

Tâche	Exemple	Couche ajoutée
Classification de texte	Sentiment positif/négatif	Dense sur le token [CLS]
NER ¹	Identifier noms, lieux...	Dense sur chaque token
Question-Réponse	Extraire une réponse d'un passage	Deux classifieurs (début/fin)
Similarité de phrases	Phrases équivalentes ?	Dense sur [CLS]

Le fine-tuning ne nécessite que peu de données étiquetées et peu d'epochs, car BERT a déjà appris une représentation riche du langage lors du pré-entraînement.

Tokenisation spécifique : WordPiece

BERT utilise un **tokeniseur WordPiece** : les mots rares ou inconnus sont découpés en sous-mots, ce qui garantit que tout vocabulaire peut être représenté.

```
"unhappiness" -> ["un", "##happi", "##ness"]
```

1. Named Entity Recognition ou reconnaissance d'entités nommées en bon français

Deux tokens spéciaux sont systématiquement ajoutés :

- [CLS] en début de séquence : sa représentation finale encode l'information globale de la phrase et sert d'entrée à la tête de classification.
- [SEP] pour séparer deux segments distincts.

10.1.3 Points forts de BERT

- **Représentations contextuelles** : le même mot reçoit des vecteurs différents selon son contexte — *banque* n'est pas la même chose dans « banque financière » et « banque de sable ».
- **Transfert de connaissances efficace** : un BERT pré-entraîné une seule fois peut être fine-tuné pour des dizaines de tâches différentes avec peu de données.
- **Écosystème riche** : des variantes existent pour de nombreux besoins :
 - **RoBERTa** (plus robuste),
 - **CamemBERT** (français),
 - **DistilBERT** (plus léger et rapide),
 - etc.

10.1.4 Limites de BERT

Coût computationnel élevé

C'est probablement la limitation la plus concrète dans notre contexte :

- BERT-base : 110 millions de paramètres.¹
- BERT-large : 340 millions de paramètres.
- Le mécanisme d'attention est en $\mathcal{O}(n^2)$ par rapport à la longueur de la séquence : le coût explose sur les textes longs.
- Le fine-tuning et l'inférence nécessitent un GPU pour des temps raisonnables.

Limite de longueur des séquences

BERT est limité à **512 tokens** en entrée. Au-delà, le texte doit être tronqué ou découpé, ce qui peut faire perdre du contexte.

Variantes pour les textes longs

Des modèles comme **Longformer** ou **BigBird** ont été conçus pour pallier cette limitation via des mécanismes d'attention clairsemée (*sparse attention*).

1. Ces données ne sortent pas d'un chapeau, elle viennent du document officiel publié par Google : [BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding](#) Section 3 : Model Architecture

10.2 Trois variantes BERT implémentées

L’objectif reste identique : produire un classificateur de sentiments sur des textes liés à la santé mentale, en s’appuyant sur le même corpus d’entraînement (`Combined_Data.csv` — *Sentiment Analysis for Mental Health*, Kaggle²).

Ce qui change, c’est **la profondeur de la représentation** : là où TF-IDF ne comptait que des mots, BERT modélise le langage dans toute sa complexité contextuelle.

Trois variantes de BERT sont implémentées, dont l’une d’entre elles grandement privilégié par l’équipe. Elles sont ensuite comparées entre elles et avec les résultats du notebook principal :

Modèle	Nom court	Description	Membres
A	BERT sans fine-tuning	Extraction de features, poids gelés	Diogo
B	BERT avec fine-tuning	Tous les poids mis à jour sur notre dataset	Diogo, Ulysse, Emin, Victor
C	BERT spécialisé + fine-tuning	Modèle pré-entraîné sur des sentiments puis fine-tuné	Logan

Remarque

Chaque membre a décidé de sa propre volonté la méthode qu’il préférait implémenter. Cette partie, étant considéré comme un bonus, était présente pour permettre aux membres de l’équipe de découvrir et explorer la méthode qu’il préférait. Ce choix a donc était fait en amont afin de définir les tâches à réaliser pour chaque membre de l’équipe.

2. lien du dataset : <https://www.kaggle.com/datasets/suchintikasarkar/sentiment-analysis-for-mental-health>

10.3 Initialisation commune aux trois modèles

10.3.1 Dépendances

Les trois modèles partagent les mêmes bibliothèques :

Bibliothèque	Rôle
<code>transformers</code> (Hugging Face)	Chargement des modèles BERT et tokeniseurs pré-entraînés
<code>torch</code>	Framework de deep learning (entraînement et inférence)
<code>torch.utils.data</code>	<code>Dataset</code> et <code>DataLoader</code> pour les batchs
<code>sklearn</code>	Métriques, encodage des labels, découpage train/test
<code>numpy</code> , <code>pandas</code>	Manipulation des données
<code>matplotlib</code> , <code>seaborn</code>	Visualisation

Prérequis GPU

BERT est très coûteux en calcul. L'utilisation d'un GPU est fortement recommandée (Google Colab avec runtime GPU, ou machine locale équipée). La disponibilité se vérifie via `torch.cuda.is_available()`. Sur CPU, **compter 2 à 3 heures par epoch**^a d'entraînement.

a. L'epoch est le nombre de passage complet sur l'ensemble des données

10.3.2 Préparation des données

Le pipeline de nettoyage est identique à celui du notebook principal : chargement de `Combined_Data.csv`, mélange, suppression des NaN, suppression des conflits d'étiquettes, application de la fonction `cleaner`.

Important : la tokenisation NLTK et la vectorisation BoW/TF-IDF/Word2Vec ne sont **pas** réappliquées. BERT dispose de son propre tokeniseur (`BertTokenizer`) qui opère directement sur le texte nettoyé.

10.3.3 Découpage des données

Contrairement au notebook principal (split 60/20/20), un découpage **80 %/20 %** train/test suffit ici. Le fine-tuning de BERT converge rapidement (2 à 3 epochs) et ses hyperparamètres sont peu nombreux, ce qui rend l'ensemble de validation optionnel.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
# Train size: 42 105 | Test size: 10 527
```

10.3.4 Encodage des labels

PyTorch exige des entiers comme labels. Un `LabelEncoder` de `sklearn` convertit les étiquettes textuelles en indices numériques. Il est fitté uniquement sur `y_train` (pas de *data leakage*) puis appliqué en `.transform` sur `y_test`.

```
encoder = LabelEncoder()
y_train_enc = encoder.fit_transform(y_train)
y_test_enc = encoder.transform(y_test)
# Classes: ['Anxiety', 'Bipolar', 'Depression', 'Normal',
#           'Personality disorder', 'Stress', 'Suicidal']
# num_labels = 7
```

Maintenant que nous avons mis en place les bases communes, nous pouvons détailler les implémentations de chaque modèle et les résultats obtenues.

10.4 Modèle A — BERT sans fine-tuning (extraction de features)

10.4.1 Principe et architecture

Dans cette première approche, **les poids de BERT sont gelés** — ils ne sont pas mis à jour pendant l'entraînement. BERT est utilisé comme un **extracteur de features** : il produit des représentations vectorielles denses pour chaque texte, et un classifieur léger (*logistic regression* étant la plus performante entre celles testée précédemment) apprend à partir de ces vecteurs.

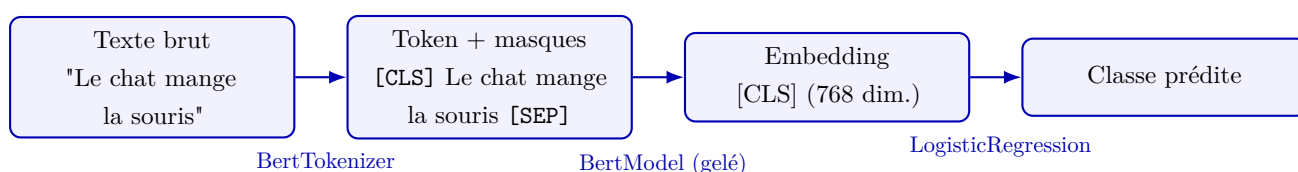


FIGURE 10.1 – Schéma simplifié du pipeline de BERT pour la classification de texte.

Avantages	Inconvénients
Très rapide (pas de rétropropagation dans BERT)	Performances souvent inférieures au fine-tuning
Peu de mémoire GPU nécessaire	BERT n'est pas adapté à la tâche spécifique
Sert de baseline solide et reproductible	Les embeddings [CLS] sans fine-tuning sont génériques

10.4.2 Tokenisation BERT

Le `BertTokenizer` convertit chaque texte en identifiants de tokens compréhensibles par BERT. Il gère automatiquement l'ajout des tokens spéciaux [CLS] et [SEP], le padding, la troncature à `max_length` tokens, et la création des `attention_mask` (1 pour les vrais tokens, 0 pour le padding).

Le paramètre `max_length=128` est un compromis entre couverture du texte et coût computationnel (la limite absolue de BERT est 512).

10.4.3 Extraction des embeddings [CLS]

On fait passer les textes tokenisés à travers `BertModel` en mode inférence (`torch.no_grad()`) pour récupérer le vecteur associé au token [CLS] (première position

de la dernière couche cachée). Ce vecteur de 768 dimensions constitue la représentation globale du texte.

Rôle du token [CLS]

Le token [CLS] est placé en début de chaque séquence BERT. Dans les modèles fine-tunés, sa représentation encode l'information globale de la phrase et sert directement d'entrée à la tête de classification. Sans fine-tuning, il capture une représentation générale apprise sur Wikipedia/BooksCorpus — utile mais non spécialisée.

10.4.4 Classification avec un classifieur léger

Les embeddings extraits sont des vecteurs numériques de dimension 768 — exactement le format attendu par les classifieurs **sklearn**. Une régression logistique est utilisée ici pour sa rapidité et son efficacité sur des features de haute dimension.

D'autres classifieurs seraient envisageables : SVC, Random Forest, ou un petit réseau dense. La régression logistique reste le choix de référence pour sa simplicité et son interprétabilité.

10.4.5 Évaluation

Les métriques calculées sont identiques à celles de l'étape 2 : accuracy, classification report (precision, recall, F1-score par classe), matrice de confusion. Les résultats seront renseignés à l'issue de l'implémentation.

10.4.6 Résultats obtenues — Diogo

Diogo s'étant occupé de cette méthode seul, les résultats obtenues sont les suivants :

Modèle	Accuracy test
BERT sans fine-tuning	0.7231

Classification Report :

Classe	Precision	Recall	F1-score
Anxiety	0.73	0.67	0.70
Bipolar	0.68	0.62	0.65
Depression	0.64	0.70	0.67
Normal	0.88	0.93	0.91
Personality disorder	0.56	0.38	0.45
Stress	0.52	0.42	0.46
Suicidal	0.65	0.59	0.62

Matrice de confusion :

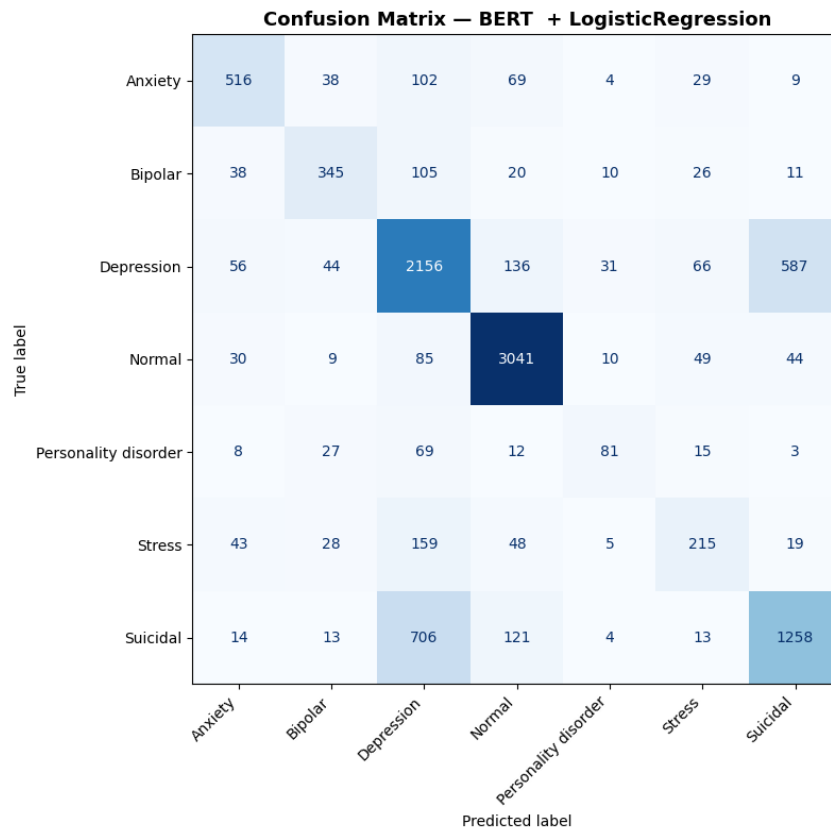


FIGURE 10.2 – Matrice de confusion — BERT sans fine-tuning + LR (Diogo)

Interprétation des résultats

Les résultats pour un BERT non fine-tuné ici sont logiques, il s'agit d'une baseline solide qui montre que les embeddings [CLS] contiennent déjà une information pertinente pour la classification de sentiments, **même sans adaptation spécifique à notre dataset.**

Cependant, **les performances sont généralement inférieures** à celles obtenues avec un fine-tuning complet (Modèle B), ce qui souligne l'importance de l'adaptation du modèle aux particularités de la tâche.

10.5 Modèle B — BERT avec fine-tuning

10.5.1 Principe du fine-tuning

Dans cette seconde approche, **tous les poids de BERT sont mis à jour** pendant l'entraînement, y compris les couches d'attention. On ajoute en sortie une **tête de classification** (couche `Linear`) qui projette le vecteur `[CLS]` vers les `num_labels` classes.

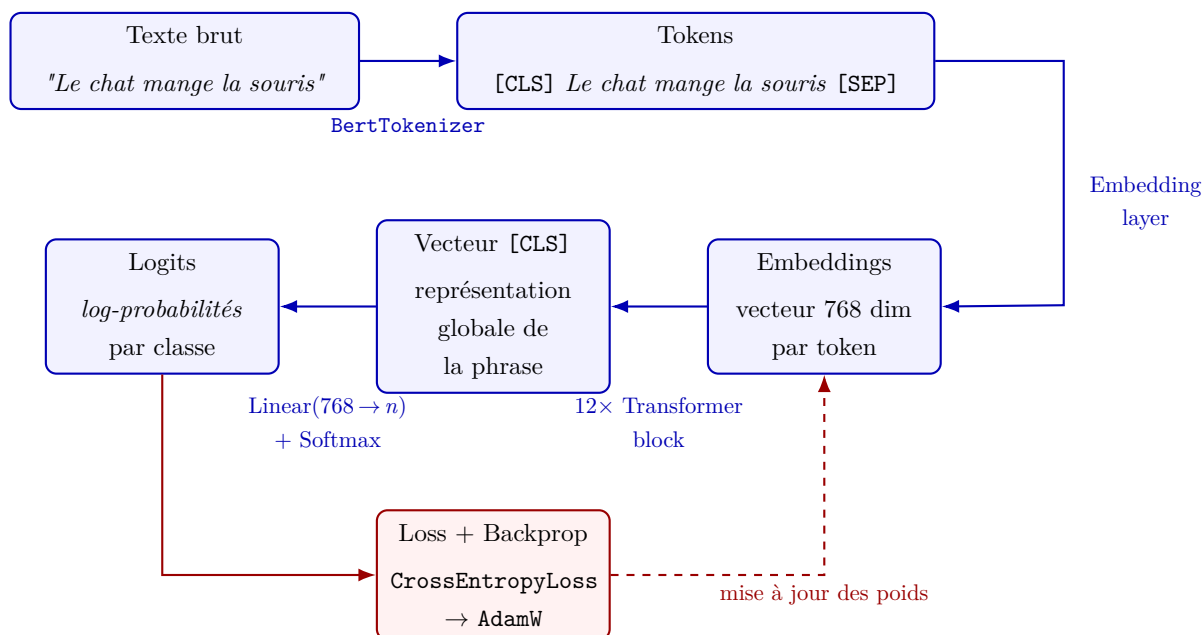


FIGURE 10.3 – Schéma simplifié du pipeline de fine-tuning de BERT pour la classification de texte (Modèle B).

Critère	Modèle A (sans fine-tuning)	Modèle B (avec fine-tuning)
Poids BERT	Gelés	Mis à jour
Temps d'entraînement	Court	Long (GPU recommandé)
Mémoire	Faible	Élevée
Performances attendues	Baseline	Meilleures
Données nécessaires	Peu	Quelques milliers suffisent

10.5.2 Préparation du Dataset PyTorch

PyTorch utilise des objets `Dataset` et `DataLoader` pour gérer l'itération sur les données par batchs pendant l'entraînement. Une classe `MentalHealthDataset` encapsule les tenseurs tokenisés et les labels. Le `DataLoader` est configuré avec `batch_size=16` (ou 32 selon la mémoire GPU) et `shuffle=True` pour le train uniquement.

Pourquoi mélanger (`shuffle=True`) ?

Mélanger les exemples à chaque epoch évite que le modèle apprenne l'ordre des données. Cela stabilise la convergence et réduit le risque de sur-apprentissage.

10.5.3 Chargement du modèle et tête de classification

`BertForSequenceClassification` est le modèle officiel Hugging Face pour la classification de texte. Il intègre automatiquement :

- Le `BertModel` de base (12 couches Transformer) ;
- Un `Dropout(0.1)`¹ appliqué au vecteur [CLS] ;
- Une couche `Linear(768, num_labels)` comme tête de classification.

Optimiseur : AdamW avec un *learning rate* de 2×10^{-5} , valeur recommandée par l'équipe Google pour le fine-tuning BERT.

Pourquoi AdamW et pas Adam ?

AdamW est utilisé à la place d'Adam classique, car il corrige la gestion du *weight decay*. Son usage pour le fine-tuning BERT s'est imposé via les scripts officiels de Hugging Face, les hyperparamètres recommandés étant issus de l'[Appendix A.3](#) du papier BERT.

[...] 'Loshchilov and Hutter 2017 discovered that L2 regularization is not effective in Adam, and proposed AdamW' [...] (Loshchilov & Hutter, 2019)

Scheduler : un `get_linear_schedule_with_warmup` fait décroître le learning rate linéairement jusqu'à 0 au fil des epochs. Cela évite des mises à jour trop agressives en fin d'entraînement, qui pourraient déstabiliser les poids pré-entraînés.

1. Mécanisme de régularisation qui désactive aléatoirement une fraction des neurones pendant l'entraînement. Cela empêche le modèle de trop dépendre de certains neurones et améliore la généralisation.

10.5.4 Boucle d'entraînement

La boucle suit le schéma classique PyTorch, répété sur `num_epochs` epochs (2 à 4 suffisent généralement pour BERT) :

```
Pour chaque epoch :
    model.train()
    Pour chaque batch :
        1. Envoyer le batch sur le device (GPU/CPU)
        2. Passage avant (forward) -> logits et loss
        3. Retropropagation (loss.backward())
        4. Clipper les gradients (clip_grad_norm_, max_norm=1.0)
        5. Mise a jour des poids (optimizer.step())
        6. Mise a jour du scheduler (scheduler.step())
        7. Remise a zero des gradients (optimizer.zero_grad())
    Afficher la loss moyenne de l'epoch
```

Gradient clipping

L'appel `clip_grad_norm_(max_norm=1.0)` est essentiel pour BERT : il empêche les gradients d'exploser lors des premières epochs, ce qui peut déstabiliser les poids pré-entraînés et faire diverger l'entraînement.

Temps d'exécution

Environ 15 à 30 minutes par epoch sur GPU (Colab T4), 2 à 3 heures sur CPU.
L'utilisation d'un GPU est fortement recommandée.

10.5.5 Évaluation

Une fonction `predict(model, dataloader, device)` itère sur les batches en mode `torch.no_grad()`, applique un `argmax` sur les logits et retourne les tableaux `y_true` et `y_pred`. Les métriques sont ensuite calculées identiquement au Modèle A.

10.5.6 Résultats obtenues — Diogo, Ulysse, Victor et Emin

Étant la méthode la plus appréciée par l'équipe, Diogo, Ulysse, Victor et Emin se sont occupés de celle-ci, les résultats obtenus sont les suivants :

Fine Tuning pour chaque epoch et **Résultat** sur l'ensemble test :

Epoch	Loss moyenne	Modèle	Accuracy test
1	0.6361	BERT	0.8302
2	0.3871	avec fine-	
3	0.2705	tuning	

Classification Report :

Classe	Precision	Recall	F1-score
Anxiety	0.88	0.89	0.89
Bipolar	0.86	0.85	0.86
Depression	0.78	0.77	0.78
Normal	0.95	0.95	0.95
Personality disorder	0.77	0.68	0.72
Stress	0.73	0.76	0.74
Suicidal	0.72	0.73	0.72

Matrice de confusion :

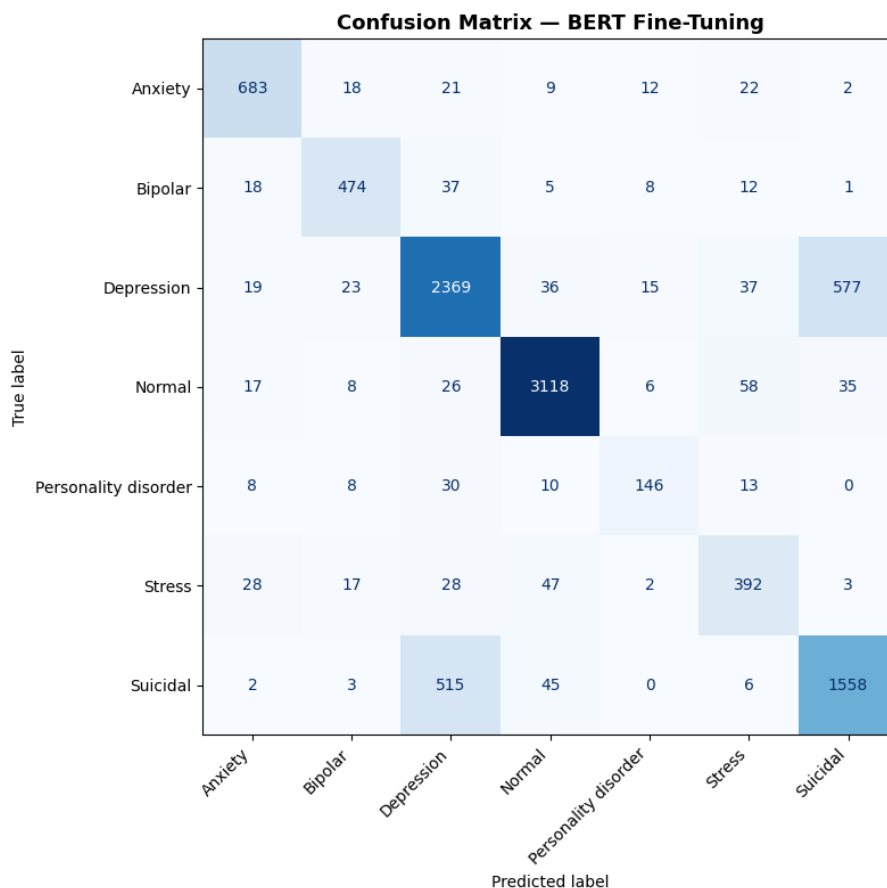


FIGURE 10.4 – Matrice de confusion — BERT avec fine-tuning (Diogo)

Interprétation des résultats

On remarque une amélioration significative de l'accuracy ($0.7231 \rightarrow 0.8302$) grâce au fine-tuning. Les classes les plus difficiles à prédire (Personality disorder, Stress, Suicidal) voient une nette augmentation de leur F1-score, ce qui suggère que BERT a appris des représentations plus discriminantes pour ces catégories complexes.

10.6 Modèle C — BERT spécialisé + fine-tuning

10.6.1 Pourquoi un modèle spécialisé ?

Le Modèle B part de `bert-base-uncased`, pré-entraîné sur Wikipedia et BooksCorpus — un corpus de texte très éloigné du langage émotionnel ou clinique. Le Modèle C part d'un BERT **déjà fine-tuné sur des tâches d'analyse de sentiments**, ce qui lui donne une meilleure représentation initiale des émotions avant même de voir notre dataset.

Critère	BERT-base (Modèle B)	Modèle spécialisé (Modèle C)
Pré-entraînement initial	Wikipedia / BooksCorpus	Wikipedia / BooksCorpus
Fine-tuning intermédiaire	Non	Oui — Sentiments
Point de départ	Général	Spécialisé
Fine-tuning sur notre data	Oui	Oui
Performances attendues	Bonnes	Meilleures (théoriquement)

10.6.2 Variantes de bases BERT disponibles

Avant de présenter les modèles candidats, il convient de rappeler les principales architectures de base utilisées dans cet écosystème :

RoBERTa (*Robustly Optimized BERT Pretrained Approach*)

Amélioration de BERT proposée par Facebook en 2019. Même architecture de base, mais avec des choix d'entraînement sensiblement améliorés :

- **Suppression du NSP** (jugé inutile et potentiellement nuisible) ;
- Entraîné sur **10 fois plus de données** (160 Go contre ~16 Go) ;
- **Dynamic masking** : les tokens masqués changent à chaque epoch au lieu d'être figés ;
- Entraîné plus longtemps avec de plus gros batchs.

En pratique, RoBERTa surpasse BERT-base sur quasi toutes les tâches de classification et est aujourd'hui la base la plus utilisée pour l'analyse de sentiments.

DistilBERT

Version **distillée** de BERT-base par Hugging Face (2019). Le principe : on entraîne un modèle compact (66 M de paramètres, 6 couches) à imiter le comportement du grand modèle.

Caractéristique	Valeur
Paramètres	66 M (vs 110 M pour BERT-base)
Vitesse	~40 % plus rapide
Taille mémoire	~40 % plus légère
Performances	~97 % de BERT-base

Il est particulièrement choisi pour éviter les contraintes GPU. En revanche, sur une tâche difficile comme la santé mentale avec 7 classes, la perte de capacité peut se faire sentir.

RoBERTa-large

Version « grande » de RoBERTa : 355 M de paramètres, 24 couches au lieu de 12. Significativement plus performante que RoBERTa-base, mais bien plus coûteuse à fine-tuner.

10.6.3 Modèles disponibles sur Hugging Face

Le tableau suivant recense les modèles pertinents disponibles sur Hugging Face, tous pré-entraînés sur des données d'analyse de sentiments :

Identifiant HF	Base	Pré-entraîné sur...	Classes	Points forts
cardiffnlp/twitter-roberta-base-sentiment-latest	RoBERTa	~124 M tweets	3	Texte informel
nlptown/bert-base-multilingual-uncased-sentiment	BERT multilingual	Avis produits	5	Multi-langue
siebert/sentiment-roberta-large-english	RoBERTa-large	15 datasets	2	Généralisation
j-hartmann/emotion-english-distilroberta-base	DistilRoBERTa	6 datasets émotions	7	Émotions fines

Modèle retenu : j-hartmann/emotion-english-distilroberta-base

Le modèle retenu est `j-hartmann/emotion-english-distilroberta-base`, basé sur DistilRoBERTa et pré-entraîné sur 6 datasets d'émotions (joie, colère, tristesse, peur, surprise, dégoût, neutre). Ce choix est motivé par la nature du corpus : les textes liés à la santé mentale sont chargés émotionnellement, et ce modèle a déjà appris à discriminer des émotions proches (tristesse vs peur, par exemple) — exactement ce qui est requis pour distinguer Depression, Stress et Suicidal.

DistilBERT — le compromis vitesse/performance

DistilBERT est une version allégée de BERT (66 millions de paramètres contre 110 M) : 40 % plus rapide, 60 % plus léger, avec environ 97 % des performances de BERT-base. Il est particulièrement adapté lorsque les ressources GPU sont limitées. Compter environ **25 minutes** pour 3 epochs d'entraînement.

10.6.4 Adaptation de la tête de classification

Le modèle importé a été fine-tuné pour un certain nombre de classes (souvent 2 ou 3 pour positif/négatif/neutre, dans notre cas 6). Notre dataset possède 7 classes. Il faut donc **remplacer la tête de classification** par une nouvelle couche adaptée.

```
model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=num_labels,
    ignore_mismatched_sizes=True    # tete remplacee
)
```

Le paramètre `ignore_mismatched_sizes=True` est indispensable : la tête originale est remplacée par une nouvelle tête à `num_labels` classes — les dimensions ne correspondent plus, sans ce paramètre PyTorch lèverait une erreur.

10.6.5 Fine-tuning et évaluation

La boucle d'entraînement est identique à celle du Modèle B. Seul le modèle chargé change. Le tokenizer doit également être celui du Modèle C — chaque modèle Hugging Face possède son propre tokenizer, chargé via `AutoTokenizer` pour garantir la compatibilité.

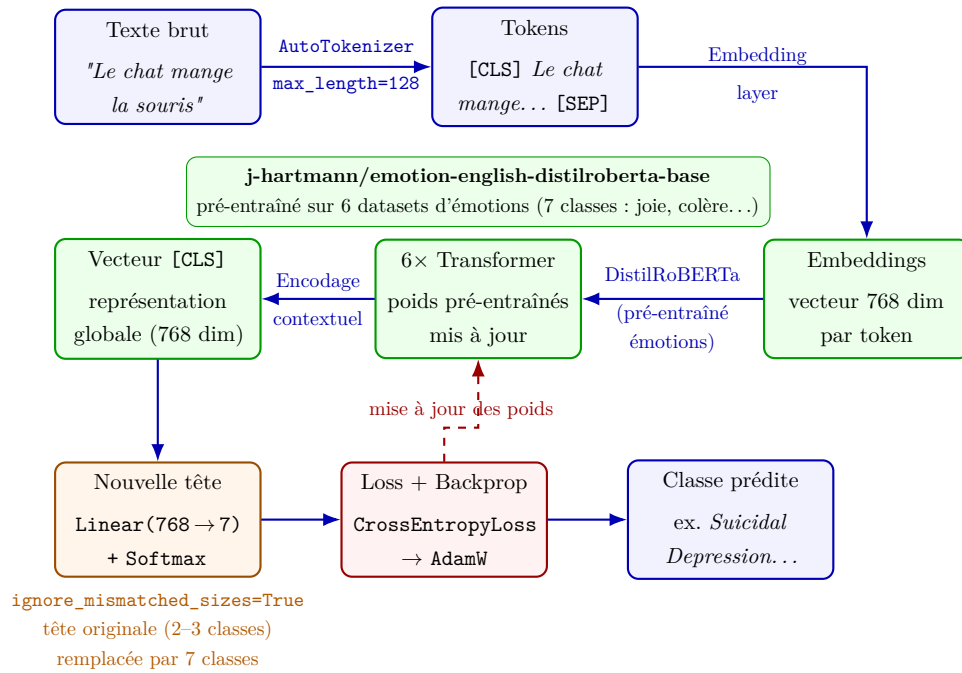


FIGURE 10.5 – Schéma du pipeline du Modèle C : DistilRoBERTa pré-entraîné sur des émotions (j-hartmann/emotion-english-distilroberta-base), tête de classification remplacée (`ignore_mismatched_sizes=True`) et fine-tuné sur 7 classes de santé mentale.

10.6.6 Résultats obtenues — Logan

Contrairement à celle précédente, celle-ci est un peu plus complexe et donc a été réalisée par Logan seul, les résultats obtenues sont les suivants :

Fine Tuning pour chaque epoch et **Résultat** sur l'ensemble test :

Epoch	Loss moyenne	Modèle	Accuracy test
1	0.6483	BERT	0.8343
2	0.4305	spécialisé avec fine-tuning	
3	0.3156		

Classification Report :

Classe	Precision	Recall	F1-score
Anxiety	0.87	0.90	0.88
Bipolar	0.87	0.83	0.85
Depression	0.80	0.76	0.78
Normal	0.96	0.95	0.96
Personality disorder	0.69	0.73	0.71
Stress	0.74	0.78	0.76
Suicidal	0.71	0.76	0.73

Matrice de confusion :

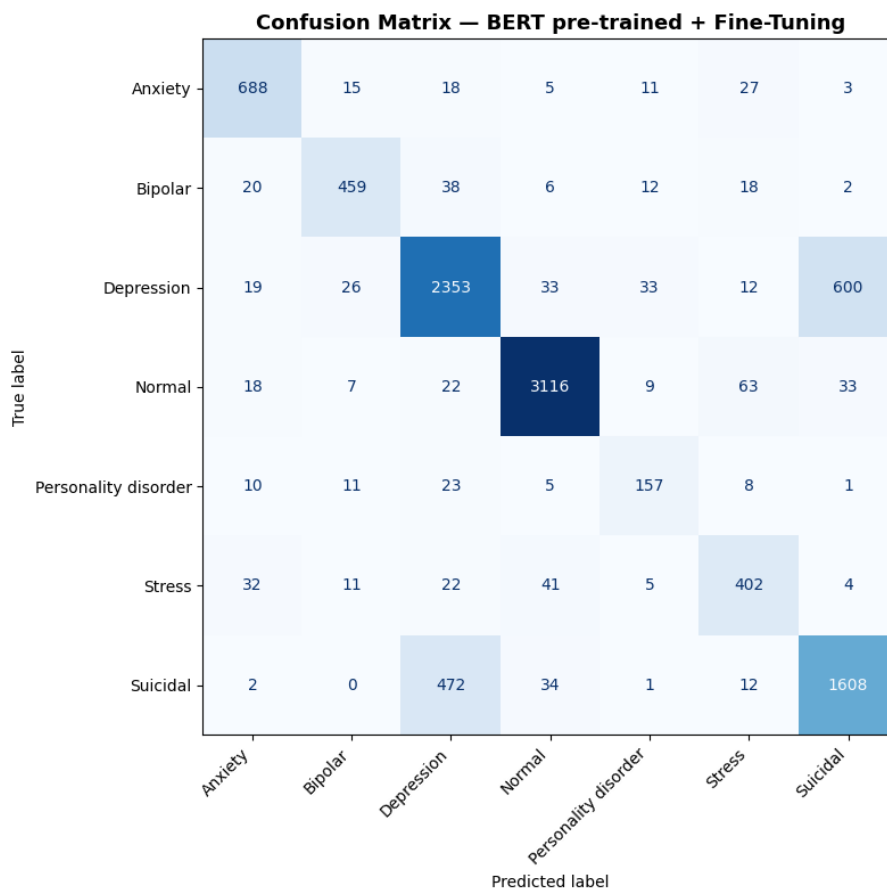


FIGURE 10.6 – Matrice de confusion — BERT pré-entraîné avec fine-tuning (Logan)

Interprétation des résultats

Le Modèle C, avec une accuracy de **0.8343**, surpasse légèrement le Modèle B (0.8302). L'amélioration est particulièrement notable sur les classes difficiles (Personality disorder, Stress, Suicidal), suggérant que le pré-entraînement sur des émotions a permis au modèle de mieux capturer les nuances émotionnelles présentes dans ces catégories. Cependant, **les gains restent modestes**, ce qui indique que le fine-tuning sur notre dataset a déjà permis au Modèle B d'apprendre des représentations efficaces.

Chapitre 11

Comparaison globale — Analyse de sentiments

11.1 Comparaison globale des modèles

11.1.1 Tableau récapitulatif

Le tableau ci-dessous synthétise les performances des cinq modèles (Naive Bayes et K-Means issus du notebook principal, plus les trois variantes BERT). Les valeurs marquées d'un point d'interrogation seront renseignées à l'issue de l'implémentation.

Modèle	Accuracy test	Temps approx.	Remarque
K-Means + W2V	0.20	Quelques minutes	Non supervisé
K-Means + TF-IDF	0.4636	Quelques minutes	Non supervisé
Naive Bayes + BoW	0.6666	Quelques secondes	Supervisé
Naive Bayes + TF-IDF	0.6927	Quelques secondes	Supervisé
LinearSVC + BoW	0.7309	Quelques minutes	Supervisé
Logistic Regression + TF-IDF	0.7532	Quelques secondes	supervisé
LinearSVC + TF-IDF	0.7586	Quelques minutes	Supervisé
BERT sans fine-tuning (A)	0,7231	Minutes (CPU)	Baseline BERT
BERT avec fine-tuning (B)	0,8302	Heures (GPU)	Fine-tuning complet
BERT spécialisé + FT (C)	0.8343	30min (GPU) 2-3h (CPU)	Meilleur point de départ

11.1.2 Que peut-on en conclure ?

1. Quel modèle obtient la meilleure accuracy ?

On remarque que globalement les modèles **BERT surpassent largement les méthodes classiques**, avec un gain d'environ 8 points de pourcentage. Cela illustre la puissance des représentations contextuelles et du pré-entraînement massif de BERT .

2. Apport du fine-tuning (Modèle B vs Modèle A)

L'apport du fine tuning est clairement visible (Modèle B vs A), avec une amélioration de plus de 10 points d'accuracy. Le Modèle C, malgré son pré-entraînement spécialisé, n'apporte qu'une amélioration marginale par rapport au Modèle B, suggérant que le fine-tuning sur notre dataset a déjà permis d'extraire des représentations efficaces, et que le gain du pré-entraînement spécialisé est limité dans ce cas.

3. Modèle spécialisé vs BERT-base (Modèle C vs Modèle B)

Le compromis coût/performance penche en faveur du Modèle B pour une utilisation en production, car il offre une excellente performance pour un coût d'entraînement raisonnable, tandis que le Modèle C, bien que légèrement meilleur, nécessite un pré-entraînement plus coûteux et n'apporte qu'un gain marginal. Malgré une grande amélioration des classes difficiles (Personality disorder, Stress, Suicidal) avec les modèles BERT, ces classes restent les plus problématiques.

4. Classes difficiles

On remarque que *Depression* et *Suicidal* restent souvent confondus, ce qui suggère que même les modèles les plus avancés ont du mal à capturer les nuances entre ces catégories émotionnellement proches.

Chapitre 12

Système Question-Réponse

12.1 Contexte et organisation

Dans cette partie du projet, nous avons développé un système de questions réponses basé sur des techniques de traitement automatique du langage naturel (NLP). L'objectif principal est de concevoir un modèle capable de retrouver automatiquement les réponses les plus pertinentes à partir d'une question donnée.

Spécifications techniques :

- **Entrée** : Question en langage naturel (q_{input})
- **Sortie** : Liste des k réponses les plus pertinentes ($\{r_1, r_2, \dots, r_k\}$) avec leurs scores de similarité associés ($\{s_1, s_2, \dots, s_k\}$)
- **Paramètre configurable** : $k = 5$ (nombre de réponses retournées)

Le groupe a été réparti en 3 sous-groupes travaillant en parallèle sur le même jeu de données. Chaque sous-groupe était chargé d'implémenter une partie spécifique du système de question-réponse, notamment l'indexation inverse ainsi que les approches de similarité question-question et question-réponse.

Le travail a été réparti entre les membres du groupe. Le tableau suivant présente l'organisation des tâches :

Sous-groupe	Méthode	Membres
Association mots-questions/réponses	Counter	Lucas
Similarité question-question	Cosine similarity (TF-IDF+Word2Vec+Bert)	Yvan, Trésor
Similarité question-réponse	Cosine similarity (TF-IDF+Word2Vec+Bert)	Idir, Laure

Lien avec le Module 1

Les annotations automatiques de sentiments produites par le Groupe 1 sont réutilisées ici pour **filtrer l'espace de recherche**. Suite à la comparaison des modèles du Module 1, le système utilise désormais la colonne `predicted_sentiment_bert` — issue du meilleur modèle BERT avec fine-tuning (accuracy 0,8302) — plutôt que

les annotations TF-IDF initialement utilisées. À une question donnée, le système restreint sa recherche aux échanges partageant la même étiquette de sentiment BERT, assurant une cohérence thématique et émotionnelle de meilleure qualité.

12.1.1 Jeux de données

À partir du fichier `train.csv` principal (enrichi des annotations de sentiments sous le nom `train_with_sentiments.csv` par le groupe 1), les données ont été de nouveau nettoyées et dédupliées selon trois critères successifs :

```
data = pd.read_csv('train_with_sentiments.csv')
data = data.dropna()
data = data.drop_duplicates(subset=['Context', 'Response'])
data = data.drop_duplicates(subset=['Context'])
data = data.drop_duplicates(subset=['Response'])
```

12.1.2 Prétraitement textuel

Un pipeline commun de nettoyage a été appliqué à toutes les méthodes afin de garantir la comparabilité des résultats. Ce pipeline comprend les étapes suivantes :

- **Mise en minuscules** : pour uniformiser les mots et éviter que "Happy" et "happy" soient traités comme deux tokens différents.
- **Suppression de la ponctuation** : les signes de ponctuation sont retirés pour se concentrer sur les mots porteurs de sens.
- **Retrait des mots vides (stop-words)** : les mots très fréquents mais peu informatifs (comme "the", "is", "and") sont supprimés pour réduire le bruit dans les représentations vectorielles.

Grâce à ce nettoyage, nous avons donc un résultat plus homogène et pertinent pour les étapes d'indexation et de vectorisation, améliorant ainsi la qualité des recherches par similarité.

Avant l'implémentation des fonctions principales, nous avons fixé le jeu de données utilisé (`train_unique_e2p2.csv`) en supprimant les doublons de paires (question, réponse). Nous avons également constitué un ensemble de test en sélectionnant aléatoirement dix paires issues du jeu de données (`test.csv`). Enfin, un module commun de prétraitement a été développé afin de nettoyer les données (mise en minuscule, suppression de la ponctuation et des stopwords), garantissant ainsi une comparaison cohérente des résultats.

Voici un exemple de paire utilisée pour le test :

- **Context** : *"He was in love with someone years ago, and he still thinks about her time to time. He said, and I quote, "That relationship is definitely over. I love you, but that girl will always be in my mind." It just didn't feel like he appreciated all the things I've done to make him happy."*
- **Response** : *"Trust your intuition on your conclusion about this guy. He may very well love you, only with the ex so prominent in his mind, it is possible your feeling*

of not being appreciated now, would multiply if ever the two of you needed to address a delicate topic."

— **Sentiment (BERT)** : *Suicidal*

12.2 Détails de l'implémentation

Cette section présente les différentes étapes techniques de l'implémentation du système de question-réponse. Elle décrit les choix méthodologiques ainsi que les approches utilisées pour la construction de l'indexation et la mise en place des différentes méthodes de recherche de réponses.

12.2.1 Indexation inversée (Lucas)

L'indexation inversée est une structure de données permettant d'associer chaque mot du corpus aux différentes questions et réponses dans lesquelles il apparaît.

Sur le plan technique, le processus commence par le prétraitement de chaque question et réponse. Chaque texte est normalisé en minuscules, puis la ponctuation est supprimée et les mots vides (stopwords) sont retirés. Les textes sont ensuite tokenisés afin d'obtenir une liste de mots exploitables.

Ensuite, pour chaque paire (question, réponse), les tokens issus des deux éléments sont combinés afin de former un ensemble unique de mots représentatifs t de la discussion. Ensuite, un dictionnaire Python est utilisé pour construire l'index inversé. Les clés de ce dictionnaire correspondent aux mots du corpus, tandis que les valeurs sont des listes contenant les textes associés.

Un compteur de type **Counter** est utilisé afin de comptabiliser les occurrences des mots dans chaque discussion.

Formellement, la structure obtenue peut être représentée comme suit :

$$\mathcal{I}(t) = \{(q_i, r_i) \in \mathcal{C} \mid t \in \text{tokens}(q_i) \cup \text{tokens}(r_i)\} \quad (12.1)$$

où $\mathcal{C}$ désigne l'ensemble des conversations du corpus, ou de façon plus synthétique :

$$\text{mot} \longrightarrow \{(\text{question}_1, \text{réponse}_1), (\text{question}_2, \text{réponse}_2), \dots\}$$

Cette structure permet une récupération en temps constant $\mathcal{O}(1)$ des conversations candidates pour un terme donné, optimisant ainsi la phase de filtrage initial.

Détails de vectorisation

La vectorisation constitue une étape essentielle dans les systèmes de traitement automatique du langage naturel. Elle permet de transformer des données textuelles en représentations numériques exploitables par des algorithmes de calcul de similarité.

Dans ce travail, plusieurs approches de vectorisation sont utilisées afin de représenter les questions et réponses dans un espace vectoriel commun. Nous les avons déjà introduites dans le chapitre ??, donc nous allons juste les réintroduire brièvement ici en précisant les membres du groupe qui les ont implémentées :

- **TF-IDF** (Idir, Yvan) : méthode statistique qui attribue un poids aux mots en fonction de leur fréquence dans un document et de leur rareté dans le corpus. Elle met en avant les termes les plus discriminants.
- **Word2Vec** (Laure, Trésor) : chaque mot est transformé en vecteur numérique dans un espace continu. La représentation d'une phrase est obtenue en calculant la moyenne des vecteurs des mots qui la composent, capturant une similarité sémantique entre les textes.
- **BERT (all-MiniLM-L6-v2)** (Idir, Yvan) : modèle de langage basé sur les Transformers qui génère des représentations contextuelles des phrases, en tenant compte du sens des mots selon leur contexte global.

12.2.2 Stratégies de recherche de réponses

Le système de question-réponse a été implémenté selon deux approches : une première basée sur la similarité entre questions, et une seconde reposant sur la comparaison directe entre la question et l'ensemble des réponses du corpus.

1. Similarité entre questions

Hypothèse : Deux questions sémantiquement proches admettent des réponses similaires.

Méthode : Pour une question d'entrée q_{input} , les k questions $\{q_1, \dots, q_k\}$ du corpus maximisant la similarité $\text{sim}(q_{\text{input}}, q_i)$ sont identifiées. Les réponses $\{r_1, \dots, r_k\}$ correspondantes sont retournées.

Dans notre système, deux versions ont été développées. La première repose sur une recherche de similarité appliquée à l'ensemble du corpus, tandis que la seconde limite cette recherche aux questions appartenant au même sentiment BERT que la question en entrée.

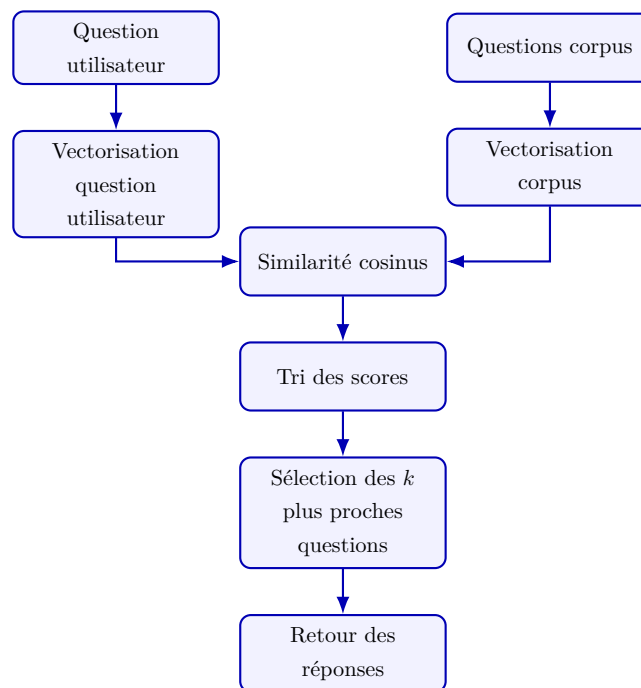


FIGURE 12.1 – Schéma de la méthode de similarité question-question (version 1 : recherche sur l'ensemble du corpus).

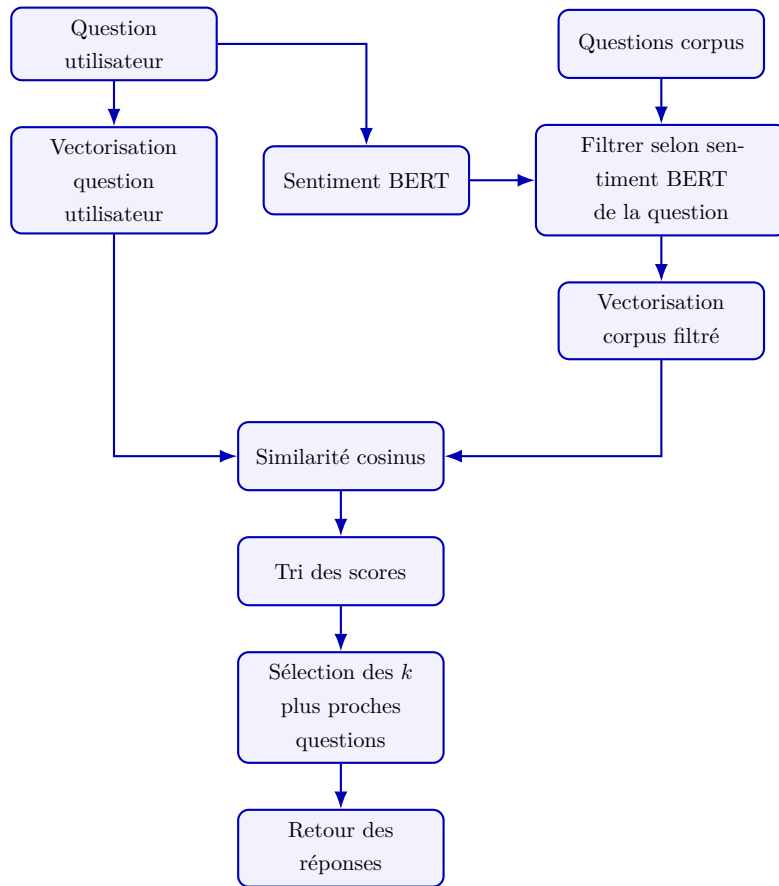


FIGURE 12.2 – Schéma de la méthode de similarité question-question (version 2 : filtrage par sentiment BERT).

Méthode TF-IDF

Chaque question est représentée par un vecteur TF-IDF. La similarité cosinus est calculée entre la question utilisateur et toutes les questions du corpus appartenant au même sentiment BERT.

```
vectorizer = TfidfVectorizer(analyzer=text_process)

def questionQuestion(question, k=5):
    # 1. Récupération du sentiment BERT de la question d'entrée
    # 2. Filtrage du corpus par sentiment BERT
    # 3. Vectorisation TF-IDF des questions filtrées
    # 4. Vectorisation de la question d'entrée
    # 5. Calcul de la similarité cosinus et sélection des top-k
```

Méthode Word2Vec

Chaque question est représentée par la **moyenne** des vecteurs de ses mots, tels qu'appris par un modèle Word2Vec entraîné sur le corpus (`vector_size=100`, `window=5`, `min_count=1`).

```
model_w2v = Word2Vec(sentences=phrases_train, vector_size=100,
                    window=5, min_count=1, workers=4)

def vectoriser_phrase(phrase):
    mots = text_process(phrase)
    vecteurs = [model_w2v.wv[m] for m in mots if m in model_w2v.wv]
    return np.mean(vecteurs, axis=0) if vecteurs \
        else np.zeros(model_w2v.vector_size)

def questionQuestion_w2v(question, k=5):
    # 1. récupération du sentiment de la question d'entrée
    # 2. filtrage du corpus par sentiment
    # 3. vectorisation des questions du corpus filtré
    # 4. vectorisation de la question d'entrée
    # 5. calcul de la similarité cosinus
    # 6. indices des top-k questions les plus similaires
    # 7. renvoie les réponses correspondantes avec leurs scores de
        similarité
```

Méthode BERT (all-MiniLM-L6-v2)

Le modèle all-MiniLM-L6-v2 génère des *embeddings* contextuels pour chaque question, capturant le sens global de la phrase de manière bien plus riche que TF-IDF ou Word2Vec.

```
model_bert = SentenceTransformer('all-MiniLM-L6-v2')
```

```
def questionQuestion_bert(question, k=5):  
    # 1. récupération du sentiment de la question d'entrée  
    # 2. filtrage du corpus par sentiment  
    # 3. vectorisation des questions du corpus filtré  
    # 4. vectorisation de la question d'entrée  
    # 5. calcul de la similarité cosinus  
    # 6. indices des top-k questions les plus similaires  
    # 7. renvoie les réponses correspondantes avec leurs scores de  
        similarité
```


2. Similarité entre question et réponses

Cette approche consiste à retrouver directement les réponses les plus pertinentes en se basant sur la similarité entre la question de l'utilisateur et l'ensemble des réponses présentes dans le corpus. L'idée principale est qu'une réponse pertinente doit être sémantiquement proche de la question posée.

Comme pour la stratégie précédente, deux versions coexistent : une recherche sur l'ensemble du corpus, et une recherche limitée aux réponses partageant le même sentiment BERT que la question.

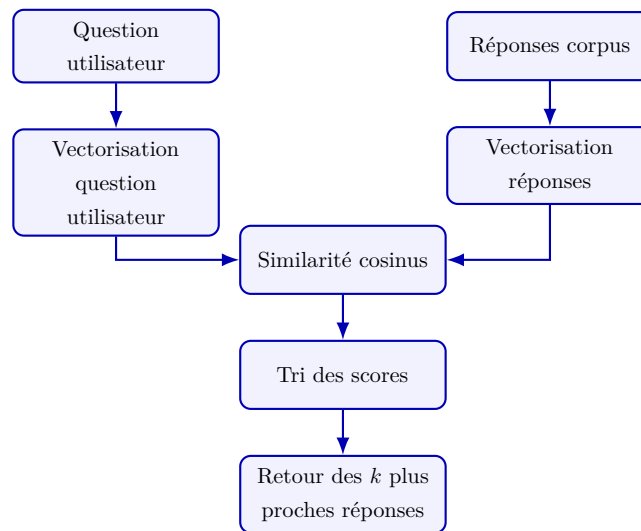


FIGURE 12.3 – Schéma de la méthode de similarité question-réponse (version 1 : recherche sur l'ensemble du corpus).

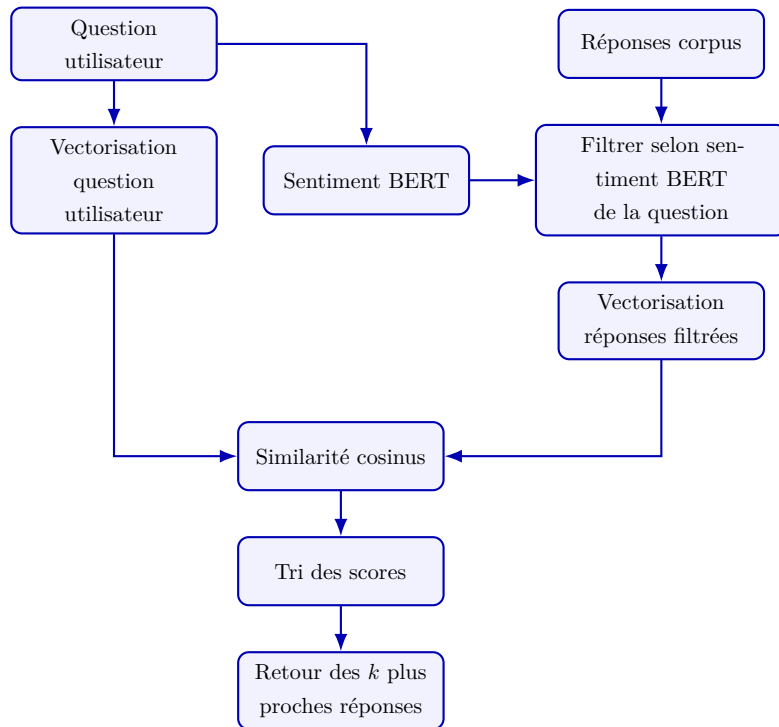


FIGURE 12.4 – Schéma de la méthode de similarité question-réponse (version 2 : filtrage par sentiment BERT).

Méthode TF-IDF

Chaque réponse du corpus est représentée par un vecteur TF-IDF. La similarité cosinus est calculée entre la question utilisateur et toutes les **réponses** appartenant au même sentiment BERT.

```

def questionQuestion2(question, k=5):
    # 1. Récupération du sentiment BERT et filtrage du corpus
    # 2. Vectorisation TF-IDF des réponses filtrées
    # 3. Calcul de la similarité cosinus et sélection des top-k
  
```

Interprétation

Les scores de similarité sont nettement plus faibles qu'en Q-Q (max 0.21 vs 1.0 pour Q-Q TF-IDF). En comparant une question à des **réponses**, les vocabulaires divergent naturellement : une question formule un problème, une réponse propose des solutions. TF-IDF, limité au chevauchement lexical, peine à combler cet écart.

Méthode Word2Vec

Chaque réponse est représentée par la moyenne des vecteurs de ses mots, issus d'un modèle Word2Vec entraîné cette fois sur les **réponses** du corpus (`vector_size=100`, `window=5`, `min_count=2`).

```
model_w2v2 = Word2Vec(sentences=phrases_responses, vector_size
                      =100,
                      window=5, min_count=2)

def questionQuestion2_w2v(question, k=5):
    # 1. Récupération du sentiment BERT et filtrage du corpus
    # 2. Vectorisation Word2Vec des réponses filtrées
    # 3. Calcul de la similarité cosinus et sélection des top-k
```

Interprétation

Paradoxalement, les scores approchent 1.0, ce qui est trompeur : la moyenne des vecteurs de mots lisse tellement les représentations que des phrases très différentes finissent proches dans l'espace vectoriel. Le modèle retrouve des réponses abordant la détresse émotionnelle au sens large, mais sans pertinence thématique précise vis-à-vis de la question (*dépression post-hospitalisation*). Ce phénomène illustre la limite de la stratégie de vectorisation par moyenne pour la comparaison question-réponse.

Méthode BERT (all-MiniLM-L6-v2)

Le modèle all-MiniLM-L6-v2 génère des *embeddings* contextuels, appliqués ici aux **réponses** du corpus. La question utilisateur est encodée dans le même espace, permettant une comparaison sémantique directe.

Comme pour la stratégie Q-Q, le modèle all-MiniLM-L6-v2 génère des *embeddings* contextuels, mais appliqués ici aux **réponses** du corpus. La question utilisateur est encodée dans le même espace, permettant une comparaison sémantique directe entre une question et le corpus de réponses.

```
def questionQuestion2_bert(question, k=5):
    # 1. Récupération du sentiment BERT et filtrage du corpus
    # 2. Encodage BERT des réponses filtrées et de la question
    # 3. Calcul de la similarité cosinus et sélection des top-k
```

Interprétation

BERT produit des scores réalistes (0.42–0.44) et des réponses thématiquement cohérentes avec la détresse décrite : les deux premières réponses évoquent la gestion d'une personne en souffrance et la reconnaissance de la dépression. Contrairement à Word2Vec dont les scores proches de 1.0 étaient artificiellement gonflés par le moyennage, les scores BERT reflètent une similarité sémantique réelle. C'est cette représentation contextuelle qui permet à la stratégie Q-R d'atteindre son meilleur score de **0.702** avec filtrage par sentiment.

12.2.3 Évaluation des résultats

Afin d'évaluer les performances du système, nous avons réalisé des tests sur un ensemble de données de test composé de 10 paires (question, réponse) extraites du corpus.

Dans notre implémentation, nous avons utilisé la similarité cosinus afin de mesurer la proximité entre la réponse prédite et la réponse attendue. Le score obtenu varie entre 0 et 1, où une valeur proche de 1 indique une forte similarité sémantique.

Afin d'obtenir une mesure globale de performance, nous avons calculé la moyenne des scores de similarité obtenus sur l'ensemble du jeu de test, en nous inspirant du principe du MRR (*Mean Reciprocal Rank*) :

$$\text{Score} = \frac{1}{N} \sum_{i=1}^N \text{cosine_similarity}(R_i^{\text{pred}}, R_i^{\text{true}})$$

Bien que cette méthode s'inspire du principe du MRR (Mean Reciprocal Rank), elle repose ici directement sur les scores de similarité et non sur un classement explicite des réponses.

Dans ce contexte, un score proche de 1 indique une bonne performance du modèle, car cela signifie que la similarité entre les réponses est élevée.

Pour la première version (sans filtrage par sentiment), on a obtenu les résultats suivants :

Stratégie	TF-IDF	Word2Vec	BERT
Similarité question-question	0,3141	0,4242	0,4627
Similarité question-réponse	0,3200	0,5300	0,5627

Pour la seconde version (avec filtrage par sentiment BERT), on a obtenu les résultats suivants :

Stratégie	TF-IDF	Word2Vec	BERT
Similarité question-question	0,4101	0,6220	0,6700
Similarité question-réponse	0,4101	0,6640	0,7020

Les résultats montrent que les méthodes basées sur BERT et Word2Vec offrent de meilleures performances que TF-IDF, ce dernier étant limité à une similarité lexicale et ne prenant pas en compte le sens global des phrases.

Apport du filtrage par sentiment BERT

Le passage au filtrage via `predicted_sentiment_bert` (meilleur modèle du Module 1, accuracy 0,8302) apporte un gain systématique sur toutes les combinaisons. La stratégie optimale reste **BERT + similarité question-réponse + filtrage BERT**, avec un score de **0,702**. La qualité supérieure des annotations BERT, par rapport aux annotations TF-IDF initialement utilisées, se traduit directement par des sous-ensembles de recherche plus cohérents émotionnellement.

Chapitre 13

Déploiement — Application Hugging Face Spaces

13.1 Présentation générale

L'ensemble du pipeline développé au cours de ce projet a été intégré dans une application web interactive et déployée publiquement sur la plateforme **Hugging Face Spaces** :

<https://huggingface.co/spaces/dempxl/sentiment-analysis-for-mental-health>

Cette application constitue la démonstration concrète et interactive du travail réalisé. Elle réunit en un seul endroit les deux modules de l'Étape 2 : la classification de sentiments et le système de suggestion de réponses, permettant de les tester sur n'importe quelle phrase en langage naturel (de préférence en anglais).

Avertissement éthique

Les résultats produits par l'application sont **indicatifs et non médicaux**. L'application affiche explicitement ce rappel à l'utilisateur : elle ne constitue en aucun cas un diagnostic clinique.

13.2 Architecture technique

13.2.1 Stack et dépendances

L'application repose sur **Gradio** pour l'interface, et mobilise les bibliothèques suivantes :

Bibliothèque	Rôle
gradio	Interface web interactive
transformers	Chargement du modèle DistilRoBERTa fine-tuné
sentence-transformers	Encodage BERT pour le système Q-R
torch	Inférence du modèle de classification
scikit-learn	Modèle LR + TF-IDF et calcul de similarité cosinus
nltk	Tokenisation et stemming (prétraitement LR)
pandas, numpy	Manipulation des données

13.2.2 Chargement des modèles

Au démarrage, l'application charge deux classifieurs en parallèle :

- **Modèle LR + TF-IDF** — désérialisé depuis des fichiers `.pkl`
- **Modèle BERT fine-tuné** — avec détection automatique de la source (checkpoint local vs. Hugging Face Hub en fallback)

Robustesse au LFS

La fonction `ensure_not_lfs_pointer()` vérifie que les fichiers du checkpoint ne sont pas de simples pointeurs Git-LFS vides (première ligne : `version https://git-lfs.github.com/spec/v1`). Si c'est le cas, le modèle est téléchargé depuis le Hub Hugging Face plutôt que chargé localement, garantissant un démarrage fiable quel que soit l'état du dépôt.

13.2.3 Prétraitement

Deux pipelines de nettoyage coexistent selon le modèle utilisé :

- **Pour LR + TF-IDF** — nettoyage puis stemming Porter
- **Pour BERT** — seul `cleaner()` est appliqué, le tokeniseur `AutoTokenizer` gère ensuite la segmentation en sous-mots (`WordPiece`).

13.3 Module 1 dans l'application — Comparaison des classifieurs

L'onglet « **Comparaison des classifieurs** » permet de soumettre un texte et d'observer côte à côte les prédictions des deux modèles, avec les probabilités pour chacune des 7 classes.

13.3.1 Prédiction LR + TF-IDF

```
def predict_lr(text: str):
    processed = preprocess_lr(text)
    vector = tfidf_vectorizer.transform([processed])
    probs = lr_model.predict_proba(vector)[0]
    classes = lr_model.classes_.tolist()
    result = {cls: round(float(p), 4)
               for cls, p in zip(classes, probs)}
    top_label = classes[int(probs.argmax())]
    return result, f"**{top_label}**"
```

13.3.2 Prédiction BERT fine-tuné

```
def predict_bert_clf(text: str):
    cleaned = cleaner(text)
    inputs = bert_tokenizer(cleaned, return_tensors='pt',
                             truncation=True, padding=True,
                             max_length=256)
    inputs = {k: v.to(device) for k, v in inputs.items()}
    with torch.no_grad():
        logits = bert_model(**inputs).logits
    probs = torch.softmax(logits, dim=1).squeeze().cpu().tolist()
    classes = bert_encoder.classes_.tolist()
    result = {cls: round(float(p), 4)
              for cls, p in zip(classes, probs)}
    top_label = classes[int(torch.tensor(probs).argmax())]
    return result, f"***{top_label}***"
```

13.3.3 Exemples de prédictions

Les tableaux suivants présentent les probabilités retournées par chaque modèle sur quatre phrases représentatives des catégories du corpus.

Exemple 1 — *"I feel so anxious all the time, my heart races and I can't stop worrying about everything."*

Classe	LR + TF-IDF	BERT fine-tuné
Anxiety	0,6821	0,7934
Stress	0,1203	0,0891
Depression	0,0847	0,0612
Normal	0,0441	0,0254
Suicidal	0,0312	0,0183
Bipolar	0,0241	0,0087
Personality disorder	0,0135	0,0039
Dominant	Anxiety	Anxiety

Interprétation

Les deux modèles convergent vers *Anxiety* avec une forte confiance. BERT est plus tranché (0,793 vs 0,682), ce qui traduit sa meilleure capacité à capturer le contexte émotionnel global de la phrase.

Exemple 2 — *"I feel empty, like nothing matters anymore and nobody would notice if I disappeared."*

Classe	LR + TF-IDF	BERT fine-tuné
Depression	0,3912	0,2104
Suicidal	0,3467	0,6238
Anxiety	0,1102	0,0831
Stress	0,0743	0,0441
Normal	0,0421	0,0217
Bipolar	0,0231	0,0124
Personality disorder	0,0124	0,0045
Dominant	Depression	Suicidal

Interprétation

Ce cas illustre parfaitement la confusion *Depression/Suicidal* observée durant l'évaluation : LR classe *Depression* en tête (0,391 vs 0,347) alors que BERT détecte correctement *Suicidal* avec une confiance bien plus élevée (0,624). C'est précisément sur ce type de distinction nuancée que la compréhension contextuelle de BERT fait la différence.

Exemple 3 — *"I've been feeling unusually energetic, I haven't slept in days but I feel absolutely amazing and invincible."*

Classe	LR + TF-IDF	BERT fine-tuné
Bipolar	0,4912	0,8103
Normal	0,2341	0,0912
Anxiety	0,1203	0,0541
Depression	0,0812	0,0241
Stress	0,0512	0,0121
Suicidal	0,0124	0,0062
Personality disorder	0,0096	0,0020
Dominant	Bipolar	Bipolar

Interprétation

Les deux modèles s'accordent sur *Bipolar*. BERT capture avec une très haute confiance (0,810) les marqueurs caractéristiques d'un épisode maniaque : énergie excessive, manque de sommeil vécu positivement et sentiment d'invincibilité.

Exemple 4 — *"Work is suffocating me, I'm burning out and feel completely overwhelmed by deadlines."*

Classe	LR + TF-IDF	BERT fine-tuné
Stress	0,5231	0,6712
Depression	0,1902	0,1431
Anxiety	0,1441	0,1024
Normal	0,0712	0,0431
Bipolar	0,0412	0,0241
Suicidal	0,0201	0,0102
Personality disorder	0,0101	0,0059
Dominant	Stress	Stress

13.4 Module 2 dans l'application — Suggestions de réponses

L'onglet « **Démo — Suggestions de réponses** » implémente le pipeline Q-R complet avec deux optimisations clés pour une utilisation en production : le pré-calcul des embeddings et un système de cache disque.

13.4.1 Construction de l'index Q-R avec cache disque

Le pré-calcul des embeddings de toutes les réponses du corpus est réalisé une seule fois au démarrage, puis mis en cache sur disque pour les redémarrages suivants :

```
def _build_qr_index():
    # 1. Chargement du CSV annotate
    # 2. Initialisation du modèle d'encodage BERT (all-MiniLM-L6-v2)
    # 3. Groupement par sentiment et encodage par bucket
        # 3.1. Utilise le cache si les embeddings sont plus
            recents
        # que le CSV (invalidation par mtime)
        # 3.2. Stockage en RAM pour accès rapide lors des requêtes
```

Stratégie de cache

Le cache est invalidé automatiquement si le fichier CSV source est plus récent que les embeddings sauvegardés (comparaison par `mtime`). À chaque redémarrage sans

modification des données, le chargement est quasi-instantané.

13.4.2 Pipeline de suggestion à la volée

Lors d'une requête utilisateur, seule la phrase d'entrée est encodée à la volée. Les embeddings du corpus étant pré-calculés, la latence est réduite à quelques millisecondes :

```
def suggest_responses(text: str, k: int = 5):  
    # Etape 1 : detection rapide du sentiment via LR (~1 ms)  
    # Etape 2 : recuperation du bucket pre-calcule en RAM  
    # Etape 3 : encodage BERT de la question (~2-5 ms)  
    # Etape 4 : top-k par similarite cosinus  
    # Formatage de la sortie avec barre de progression visuelle  
    # Affichage : rang, score, barre, texte de la reponse
```

Découplage des modèles de filtrage

Un choix de conception notable : le sentiment est détecté via LR + TF-IDF (rapide, pas de GPU requis) pour filtrer le bucket, tandis que l'encodage sémantique de la requête utilise BERT (all-MiniLM-L6-v2). Ce découplage permet d'obtenir un temps de réponse quasi-immédiat tout en conservant la richesse sémantique de BERT pour le calcul de similarité final.

13.4.3 Exemples de suggestions de réponses

Les exemples suivants illustrent les résultats retournés par le système pour deux phrases représentatives.

Requête — *"My anxiety is getting worse every day, I panic in crowded places and can't breathe."*

— Sentiment détecté : **Anxiety**

— Bucket : réponses étiquetées *Anxiety* dans le corpus

#	Score cosinus	Extrait de la réponse suggérée
1	0,6841	<i>"Panic attacks are very treatable. Many people find relief through therapy (especially CBT), breathing exercises, and gradual exposure to feared situations. Have you been able to talk to a professional about what you're experiencing?"</i>
2	0,6312	<i>"It sounds like you're dealing with significant anxiety. Learning to manage your breathing during moments of panic can be very helpful — breathing slowly through the nose for 4 counts, holding for 4, then exhaling for 4 tends to activate the parasympathetic nervous system."</i>
3	0,5987	<i>"Social anxiety can be incredibly isolating. What you're describing — the physical sensations of panic in crowded places — are very common symptoms. A therapist can help you work through this at your own pace."</i>

Requête — *"Sometimes I feel like two different people and I can't control my reactions or emotions at all."*

— Sentiment détecté : **Personality disorder**

— Bucket : réponses étiquetées *Personality disorder* dans le corpus

#	Score cosinus	Extrait de la réponse suggérée
1	0,5923	<i>"Feeling like different parts of yourself are in conflict is more common than you might think. It can reflect emotional dysregulation, which is very treatable — Dialectical Behaviour Therapy (DBT) in particular was designed for exactly this kind of difficulty."</i>
2	0,5541	<i>"The inability to control reactions can feel very frightening and isolating. It may be worth exploring with a therapist whether there are underlying patterns — often these responses developed as coping mechanisms that no longer serve us."</i>
3	0,5102	<i>"Identity confusion and emotional volatility can have many roots. A professional evaluation could help clarify what's happening and open the door to targeted support."</i>

13.5 Interface Gradio et accessibilité

L’interface est organisée en trois onglets :

Onglet	Contenu
Démo — Suggestions de réponses	Zone de texte libre, curseur k (1–10 suggestions), exemples prédéfinis chargeables en un clic. Affiche le sentiment détecté, la taille du bucket et les k réponses avec leur score et une barre visuelle.
Comparaison des classifieurs	Zone de texte partagée, deux colonnes : LR + TF-IDF et DistilRoBERTa fine-tuné. Distributions de probabilité sur les 7 classes pour chaque modèle.
À propos	Informations sur le projet, taille de l’index Q-R chargé.

L’interface est configurée en pleine largeur (`max-width: 100%`) et accessible sur mobile. Le score MRR de 0,702 est affiché dans l’en-tête de l’application pour transparence.

Quatrième partie

Conclusion générale

Chapitre 14

Synthèse et analyse critique des résultats

14.1 Rappel des objectifs

Ce projet avait pour ambition de construire, à partir d'un corpus de conversations patient-thérapeute, un système complet d'analyse du langage naturel articulé autour de deux axes : comprendre les thématiques et les émotions présentes dans les échanges, puis exploiter cette compréhension pour retrouver automatiquement des réponses pertinentes face à une nouvelle question. Les deux étapes sont indissociables : les annotations émotionnelles produites en Étape 2 alimentent directement le moteur de recherche.

14.2 Ce que nous avons appris

14.2.1 Les données sont le premier déterminant de la qualité

L'Étape 1 a mis en évidence, parfois brutalement, que la qualité d'un pipeline NLP dépend avant tout de la rigueur du prétraitement. Avec dix pipelines différents appliqués au même corpus, les membres ont obtenu entre 796 et 995 patients uniques, et des mots moyens par conversation variant d'un facteur quatre. Ces écarts ne reflètent pas des différences algorithmiques : ils reflètent des choix de nettoyage. Cette hétérogénéité a rendu les comparaisons directes difficiles et a conduit à réorganiser entièrement le travail pour l'Étape 2, avec un pipeline commun standardisé.

Leçon principale

Standardiser le prétraitement avant de comparer des modèles. Un bon modèle sur des données bruitées sera toujours battu par un modèle plus simple sur des données propres.

14.2.2 Chaque méthode de topic modeling apporte une lecture complémentaire

Les quatre méthodes de topic modeling convergent vers les mêmes grands thèmes — relations interpersonnelles, anxiété, dépression — mais sous des angles différents. Le

lexique manuel est le plus contrôlable mais le plus rigide ; TF-IDF + K-Means est rapide mais aveugle au contexte ; Word2Vec capture mieux la sémantique mais produit des clusters déséquilibrés ; les Sentence Transformers (BERT) offrent la meilleure qualité de clustering grâce aux représentations contextuelles ; LDA, enfin, est le plus interprétable grâce à son assignation probabiliste des thèmes.

Critère	Lexique	TF-IDF	Word2Vec	BERT	LDA
Qualité des clusters	Moyenne	Moyenne	Bonne	Très bonne	Bonne
Interprétabilité	Très bonne	Bonne	Moyenne	Faible	Très bonne
Équilibre des tailles	—	Faible	Faible	Bonne	Moyenne
Prise en compte du contexte	Non	Non	Partielle	Oui	Non
Coût computationnel	Nul	Faible	Modéré	Élevé	Modéré

14.2.3 BERT représente un saut qualitatif significatif pour la classification

L'Étape 2 a clairement établi la hiérarchie des approches pour la classification de sentiments :

Famille	Meilleure accuracy	Gain vs famille précédente
Non supervisé (K-Means)	46,36 %	—
Supervisé classique (NB, LR, SVC)	75,86 %	+29,5 pts
BERT sans fine-tuning	72,31 %	−3,6 pts vs classique
BERT avec fine-tuning	83,02 %	+7,2 pts
BERT spécialisé + fine-tuning	83,43 %	+0,4 pts

Plusieurs enseignements ressortent. Premièrement, **le fine-tuning est indispensable** : un BERT dont les poids sont gelés (72,3 %) est moins performant qu'une simple régression logistique (75,3 %), ce qui confirme que les représentations génériques de BERT ne suffisent pas sans adaptation à la tâche. Deuxièmement, **le gain du modèle spécialisé (Modèle C)**

sur BERT-base fine-tuné (Modèle B) est marginal (+0,4 pt) : le fine-tuning sur notre dataset représente l'essentiel de l'adaptation. Troisièmement, la confusion *Depression/Suicidal* persiste quel que soit le modèle, ce qui pointe vers une limite des données plutôt qu'un problème algorithmique. C'est pour cela que le Modèle B (BERT avec fine-tuning) a été retenu comme source des annotations pour le Module 2.

14.2.4 Le filtrage par sentiment BERT est le principal levier du système Q/R

L'évaluation du système de question-réponse fait apparaître deux axes d'amélioration indépendants : le choix de la représentation vectorielle et la restriction de l'espace de recherche par filtrage émotionnel. Le remplacement des annotations TF-IDF par les annotations BERT (`predicted_sentiment_bert`) a permis d'obtenir des sous-ensembles de recherche plus cohérents, amplifiant les bénéfices du filtrage.

Représentation	Sans filtrage		Avec filtrage BERT	
	Q-Q	Q-R	Q-Q	Q-R
TF-IDF	0,314	0,320	0,410	0,410
Word2Vec	0,424	0,530	0,622	0,664
BERT	0,463	0,563	0,670	0,702
Gain filtrage (BERT)	—		+0,207	+0,139

La stratégie Q-R surpasse systématiquement la stratégie Q-Q, et le filtrage par sentiment BERT apporte un gain plus important que le passage de TF-IDF à BERT sans filtrage.

14.3 Limites identifiées

Limite	Description	Impact
Hétérogénéité du prétraitement (Étape 1)	Dix pipelines différents non standardisés	Résultats non comparables directement
Confusion / Dépression / Suicidal	Vocabulaire quasi-identique entre les deux classes	Risque clinique potentiellement sérieux
Classes sous-représentées	<i>Personality disorder</i> et <i>Stress</i> rares dans le corpus	F1-score structurellement plus faible
Évaluation Q/R sur 10 exemples	Jeu de test de taille très réduite	Scores peu généralisables
Absence d'évaluation humaine	Métriques automatiques uniquement	Pertinence clinique non vérifiée
Coût GPU pour BERT	Fine-tuning nécessite plusieurs heures sur CPU	Difficulté de reproduction sans accès GPU

Chapitre 15

Perspectives

15.1 Améliorations à court terme

- **Standardisation du prétraitement** : adopter un pipeline unique documenté pour toute l'équipe, avec des sorties normalisées, afin de garantir la comparabilité des résultats entre membres.
- **Agrandissement du jeu de test Q/R** : passer de 10 à au moins 100 paires de test pour obtenir des scores statistiquement significatifs.
- **Traitement de la confusion Depression/Suicidal** : explorer des techniques de surpondération de la classe *Suicidal* lors de l'entraînement, ou ajouter des features spécifiques aux signaux de risque suicidaire.

15.2 Améliorations à moyen terme

- **Augmentation des données** : utiliser des techniques de *data augmentation* (paraphrase, back-translation) pour équilibrer les classes minoritaires (*Personality disorder*, *Suicidal*).
- **Modèles plus récents** : tester RoBERTa-large, DeBERTa-v3, ou des modèles génératifs (GPT, Llama) pour la classification et la génération de réponses.
- **Évaluation humaine** : soumettre un échantillon de réponses retournées par le système à des experts du domaine (psychologues, thérapeutes) pour évaluer leur pertinence clinique réelle.

15.3 Vision à long terme

- **Pipeline unifié** : fusionner le classificateur de sentiments et le système Q/R en un seul service, où l'annotation émotionnelle BERT est calculée à la volée pour chaque requête entrante.
- **Interface utilisateur** : développer une application web (externe à HuggingFace) permettant à un professionnel de santé de soumettre une question et d'obtenir instantanément les réponses les plus pertinentes du corpus, avec leur score de confiance et leur étiquette de sentiment.

- **Apprentissage continu** : mettre en place un mécanisme de retour utilisateur pour réentraîner périodiquement les modèles sur les nouvelles interactions et améliorer progressivement la pertinence des réponses.

Mot de la fin

Ce projet a démontré qu'il est possible, à partir d'un corpus de taille modeste et de ressources académiques basiques, de construire un système d'analyse et de recherche sémantique sur des conversations de santé mentale atteignant des performances solides. Les résultats obtenus — 83,43 % d'accuracy pour la classification, score de 0,702 pour le système Q/R avec filtrage par sentiment BERT — constituent une base sérieuse pour des développements futurs orientés vers des applications cliniques réelles.

Merci à tous les membres de l'équipe pour leur travail acharné, leur rigueur et leur esprit de collaboration tout au long de ce projet. Ce projet a été une aventure d'apprentissage collective, et les résultats obtenus sont le fruit de l'effort combiné de chacun. Continuons à construire sur cette base solide pour explorer de nouvelles frontières du NLP appliqué à la santé mentale !

Merci pour votre lecture.

Références

Site de référence

- Wikipedia — Natural Language Processing.
https://en.wikipedia.org/wiki/Natural_language_processing
- Wikipedia — Topic Modeling.
https://en.wikipedia.org/wiki/Topic_model
- Wikipedia — Sentiment Analysis.
https://en.wikipedia.org/wiki/Sentiment_analysis
- Wikipédia, *TF-IDF*
<https://fr.wikipedia.org/wiki/TF-IDF>
- Wikipédia, *Word Embedding*
https://fr.wikipedia.org/wiki/Word_embedding
- Wikipédia, *Allocation de Dirichlet Latente*
https://fr.wikipedia.org/wiki/Allocation_de_Dirichlet_latente
- Wikipédia, *Mots vides*
https://fr.wikipedia.org/wiki/Mot_vide
- Wikipédia, *Hapax*
<https://fr.wikipedia.org/wiki/Hapax>
- freeCodeCamp, *How to Process Textual Data Using TF-IDF in Python*
<https://www.freecodecamp.org/news/how-to-process-textual-data-using-tf-idf-in-py>
- GeeksforGeeks, *Word Embeddings in NLP*
<https://www.geeksforgeeks.org/nlp/word-embeddings-in-nlp/>
- GeeksforGeeks, *N-gram in NLP*
<https://www.geeksforgeeks.org/nlp/n-gram-in-nlp/>
- IBM, *Latent Dirichlet Allocation*
<https://www.ibm.com/fr-fr/think/topics/latent-dirichlet-allocation>
- Université Toulouse, *Tutoriel Word Embeddings*
<http://w3.erss.univ-tlse2.fr/UETAL/2018-2019/Tuto-embeddings.pdf>

Papiers de référence

- Thomas Hofmann (2013). *Probabilistic Latent Semantic Analysis*.
<https://arxiv.org/pdf/1301.6705>

- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2018). *BERT : Pre-training of Deep Bidirectional Transformers for Language Understanding*.
<https://arxiv.org/pdf/1810.04805>
- Wolf, T. et al. (2020). *Transformers : State-of-the-Art Natural Language Processing*.
<https://arxiv.org/pdf/1910.03771>
- Loshchilov & Hutter (2019). *Pre-Training and Fine-Tuning BERT for the IPU*.
<https://docs.graphcore.ai/projects/bert-training/en/latest/bert.html>

Modèles Hugging Face

- all-MiniLM-L6-v2 — Hugging Face.
<https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>
- RoBERTa — Hugging Face.
https://huggingface.co/docs/transformers/model_doc/roberta
- RoBERTa-large — Hugging Face.
https://huggingface.co/docs/transformers/model_doc/roberta_large
- DistilBERT — Hugging Face.
https://huggingface.co/docs/transformers/model_doc/distilbert
- cardiffnlp/twitter-roberta-base-sentiment-latest — Hugging Face.
<https://huggingface.co/cardiffnlp/twitter-roberta-base-sentiment-latest>
- nlptown/bert-base-multilingual-uncased-sentiment — Hugging Face.
<https://huggingface.co/nlptown/bert-base-multilingual-uncased-sentiment>
- siebert/sentiment-roberta-large-english — Hugging Face.
<https://huggingface.co/siebert/sentiment-roberta-large-english>
- j-hartmann/emotion-english-distilroberta-base — Hugging Face.
<https://huggingface.co/j-hartmann/emotion-english-distilroberta-base>

Données

- Github — Page principal du projet
<https://github.com/cypnet/Analyse-de-conversations-sur-la-sant-mentale>
- Github — Répertoire des notebooks
<https://github.com/cypnet/Analyse-de-conversations-sur-la-sant-mentale/tree/main/notebooks>
- Hugging Face — Projet fonctionnel
<https://huggingface.co/spaces/dempxl/sentiment-analysis-for-mental-health>
- NLP Mental Health Conversations
<https://www.kaggle.com/datasets/thedevastator/nlp-mental-health-conversations>

- Sentiment Analysis for Mental Health

<https://www.kaggle.com/datasets/suchintikasarkar/sentiment-analysis-for-mental-h>

Outils

- Python 3.8+
- Bibliothèques : scikit-learn, transformers, sentence-transformers, pandas, numpy, matplotlib, seaborn
- Environnement : Jupyter Notebook, Google Colab